面向国际留学生的信息管理系统V1.0源代码

一、源代码属性结构说明

1. 软件名称：面向国际留学生的信息管理系统V1.0
2. 源码根目录：当前项目根目录，源码稿仅列出相对路径，不暴露真实服务器路径。
3. 选入范围：自研前端页面、后端服务、数据库 SQL、配置文件、业务测试脚本和业务运行脚本。排除第三方依赖、构建产物、锁文件、图片、缓存、文档输出目录以及与目标软件运行无关的材料辅助工具。
4. 脱敏说明：数据库口令、令牌密钥、账号密码、邮箱、手机号、主机地址和接口地址已统一脱敏为 *** 或等价占位值。

二、三级目录结构

```
.env.example
README.md
database/schema.sql
package.json
public/app.js
public/index.html
public/styles.css
src/config/db.js
src/initDb.js
src/middleware/auth.js
src/routes/auth.js
src/routes/grades.js
src/routes/management.js
src/routes/students.js
src/routes/teachers.js
src/seedBusinessData.js
src/server.js
src/services/gradeAdapter.js
```

三、逐文件源码正文

3.1 .env.example

文件路径：.env.example

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
PORT=3000
DB_HOST=***
DB_PORT=3306
DB_USER=root
DB_PASSWORD=***
DB_NAME=international_student_platform
JWT_SECRET=***
JWT_EXPIRES_IN=8h
```

3.2 README.md

文件路径：README.md

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
# 面向国际留学生的信息管理系统 V1.0

前台使用 H5，后台使用 Node.js + Express，数据库使用 MySQL。

## 功能范围

- 学生登录与学生端中英双语界面
- 学生档案、国籍、班级、住宿、签证、学习情况基础数据模型
- 教师管理、教师班级管理、教师成绩管理入口
```

- 多国家、多学校成绩对接与成绩适配接口
- 成绩原始值、适配值、GPA、学分、规则版本、审核状态和异常原因留痕
- 管理端统计分析接口

数据库配置

默认连接配置：

host: *** port: 3306 user: root password: *** database: international_student_platform

启动步骤

1. 安装依赖：

npm install

2. 创建本地环境配置：

cp .env.example .env

3. 初始化数据库：

mysql -h *** -P 3306 -u root -p < database/schema.sql

也可以使用内置 Node 脚本初始化，不依赖本机 `mysql` 命令：

npm run db:init

4. 写入业务关联测试数据。该脚本会让每张业务表不少于 100 条记录，并保持用户、学生、教师、班级、住宿、签证、学习、外部学校、成绩规则和成绩记录之间的业务关联：

npm run db:seed

5. 启动服务：

npm start

访问地址：

http://***:3000

演示账号

三个演示账号的初始密码均为 `***`。

admin teacher01 student01 teacher001 student001

核心接口

- `POST /api/auth/login` 登录
- `GET /api/students/me` 学生查看本人档案
- `GET /api/students` 教师/管理员查看学生列表
- `GET /api/teachers` 管理员查看教师列表
- `POST /api/grades/adapt` 教师/管理员提交外部成绩并执行适配
- `GET /api/grades/student/:studentId` 查看学生成绩适配记录
- `PATCH /api/grades/:id/audit` 管理员审核成绩
- `GET /api/management/dashboard` 统计分析
- `GET /api/management/risks` 风险查询，聚合签证临期、成绩适配异常、低分风险、学习预警和住宿异常

开源协议

本项目使用 GNU General Public License v2.0（GPL-2.0）开源，详见 `LICENSE`。

3.3 database/schema.sql

文件路径：database/schema.sql

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
CREATE DATABASE IF NOT EXISTS international_student_platform
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

USE international_student_platform;

CREATE TABLE IF NOT EXISTS users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(64) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('student', 'teacher', 'admin') NOT NULL,
  display_name VARCHAR(120) NOT NULL,
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS nationalities (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  country_code VARCHAR(8) NOT NULL UNIQUE,
  name_zh VARCHAR(120) NOT NULL,
  name_en VARCHAR(120) NOT NULL,
  continent VARCHAR(60),
  primary_language VARCHAR(80),
  timezone VARCHAR(80)
);

CREATE TABLE IF NOT EXISTS teachers (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT UNIQUE,
  teacher_no VARCHAR(64) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  college VARCHAR(120),
  phone VARCHAR(40),
  email VARCHAR(120),
  teacher_type ENUM('course_teacher', 'head_teacher', 'counselor', 'admin_teacher') NOT NULL DEFAULT
'course_teacher',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_teachers_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS classes (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  code VARCHAR(64) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  college VARCHAR(120),
  major VARCHAR(120),
  teacher_id BIGINT,
  enrollment_year INT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_classes_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

CREATE TABLE IF NOT EXISTS students (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT UNIQUE,
  student_no VARCHAR(64) NOT NULL UNIQUE,
  chinese_name VARCHAR(120),
  english_name VARCHAR(120) NOT NULL,
  gender VARCHAR(20),
  birth_date DATE,
```

```
  nationality_id BIGINT,
  passport_no VARCHAR(80),
  passport_expiry DATE,
  phone VARCHAR(40),
  email VARCHAR(120),
  emergency_contact VARCHAR(160),
  enrollment_year INT,
  major VARCHAR(120),
  college VARCHAR(120),
  class_id BIGINT,
  status ENUM('active', 'suspended', 'withdrawn', 'graduated', 'delayed') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_students_user FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_students_nationality FOREIGN KEY (nationality_id) REFERENCES nationalities(id),
  CONSTRAINT fk_students_class FOREIGN KEY (class_id) REFERENCES classes(id)
);

CREATE TABLE IF NOT EXISTS dormitories (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  building VARCHAR(120) NOT NULL,
  room_no VARCHAR(60) NOT NULL,
  bed_no VARCHAR(60) NOT NULL,
  student_id BIGINT,
  check_in_date DATE,
  check_out_date DATE,
  status ENUM('available', 'occupied', 'reserved', 'maintenance') NOT NULL DEFAULT 'available',
  CONSTRAINT fk_dormitories_student FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS visas (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  visa_type VARCHAR(80) NOT NULL,
  visa_no VARCHAR(120),
  issue_date DATE,
  expiry_date DATE NOT NULL,
  residence_permit_no VARCHAR(120),
  document_url VARCHAR(255),
  status ENUM('valid', 'expiring', 'expired', 'renewing') NOT NULL DEFAULT 'valid',
  CONSTRAINT fk_visas_student FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS study_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  course_code VARCHAR(80),
  course_name VARCHAR(160) NOT NULL,
  attendance_rate DECIMAL(5,2),
  homework_status VARCHAR(120),
  class_performance VARCHAR(255),
  warning_level ENUM('none', 'notice', 'warning', 'serious') NOT NULL DEFAULT 'none',
  teacher_id BIGINT,
  recorded_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_study_records_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_study_records_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

CREATE TABLE IF NOT EXISTS external_schools (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  country_code VARCHAR(8),
  name VARCHAR(180) NOT NULL,
  contact_name VARCHAR(120),
  contact_email VARCHAR(120),
  integration_type ENUM('api', 'excel', 'csv', 'manual', 'scheduled_sync') NOT NULL DEFAULT 'manual',
  endpoint_url VARCHAR(255),
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active'
);

CREATE TABLE IF NOT EXISTS grade_rules (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
```

```
  school_id BIGINT,
  country_code VARCHAR(8),
  name VARCHAR(160) NOT NULL,
  version VARCHAR(40) NOT NULL,
  gpa_scale DECIMAL(4,2) NOT NULL DEFAULT 4.00,
  credit_ratio DECIMAL(6,3) NOT NULL DEFAULT 1.000,
  pass_score DECIMAL(5,2) NOT NULL DEFAULT 60.00,
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active',
  effective_from DATE,
  effective_to DATE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_grade_rules_school FOREIGN KEY (school_id) REFERENCES external_schools(id)
);

CREATE TABLE IF NOT EXISTS grade_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  course_code VARCHAR(80),
  course_name VARCHAR(160),
  source_school_id BIGINT,
  source_country_code VARCHAR(8),
  original_grade VARCHAR(40),
  original_type ENUM('percent', 'letter', 'gpa', 'pass_fail'),
  original_credit DECIMAL(6,2) DEFAULT 0,
  adapted_score DECIMAL(5,2),
  adapted_gpa DECIMAL(4,2),
  adapted_credit DECIMAL(6,2),
  passed TINYINT(1),
  rule_version VARCHAR(40),
  adapt_status ENUM('pending', 'success', 'exception') NOT NULL DEFAULT 'pending',
  exception_reason VARCHAR(255),
  audit_status ENUM('pending', 'approved', 'rejected', 'archived') NOT NULL DEFAULT 'pending',
  audited_by BIGINT,
  audited_at TIMESTAMP NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_grade_records_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_grade_records_school FOREIGN KEY (source_school_id) REFERENCES external_schools(id),
  CONSTRAINT fk_grade_records_auditor FOREIGN KEY (audited_by) REFERENCES users(id)
);

INSERT IGNORE INTO users (id, username, password_hash, role, display_name)
VALUES
  (1, 'admin', '***BCRYPT_HASH***', 'admin', 'System Admin'),
  (2, 'teacher01', '***BCRYPT_HASH***', 'teacher', 'Teacher Demo'),
  (3, 'student01', '***BCRYPT_HASH***', 'student', 'Student Demo');

UPDATE users
SET password_hash = '***BCRYPT_HASH***'
WHERE username IN ('admin', 'teacher01', 'student01');

INSERT IGNORE INTO nationalities (id, country_code, name_zh, name_en, continent, primary_language,
timezone)
VALUES
  (1, 'TH', '泰国', 'Thailand', 'Asia', 'Thai', 'Asia/Bangkok'),
  (2, 'VN', '越南', 'Vietnam', 'Asia', 'Vietnamese', 'Asia/Ho_Chi_Minh'),
  (3, 'MY', '马来西亚', 'Malaysia', 'Asia', 'Malay', 'Asia/Kuala_Lumpur'),
  (4, 'ID', '印度尼西亚', 'Indonesia', 'Asia', 'Indonesian', 'Asia/Jakarta');

INSERT IGNORE INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
VALUES (1, 2, 'T2026001', 'Teacher Demo', 'International Education College', '13800000000', '***@***',
'head_teacher');

INSERT IGNORE INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
VALUES (1, 'ISE-2026-A', 'International Student Class A', 'International Education College', 'Chinese
Language', 1, 2026);

INSERT IGNORE INTO students
  (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
   passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, college,
class_id, status)
```

```
VALUES
  (1, 3, 'S2026001', '学生示例', 'Student Demo', 'Female', '2004-08-01', 1,
   'P000001', '2029-08-01', '13900000000', '***@***', 'Family +66 000000', 2026,
   'Chinese Language', 'International Education College', 1, 'active');

INSERT IGNORE INTO external_schools (id, country_code, name, integration_type, status)
VALUES
  (1, 'TH', 'Demo Partner University Thailand', 'api', 'active'),
  (2, 'VN', 'Demo Partner College Vietnam', 'excel', 'active');

INSERT IGNORE INTO grade_rules (id, school_id, country_code, name, version, gpa_scale, credit_ratio,
pass_score, status)
VALUES
  (1, NULL, NULL, 'Default international grade adapter', 'default-v1', 4.00, 1.000, 60.00, 'active'),
  (2, 1, 'TH', 'Thailand partner GPA rule', 'TH-GPA-2026', 4.00, 1.000, 60.00, 'active');
```

3.4 package.json

文件路径：package.json

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
{
  "name": "southeast-asian-student-service-platform",
  "version": "1.0.0",
  "license": "GPL-2.0-only",
  "description": "Information management and service platform for Southeast Asian international students.",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js",
    "db:init": "node src/initDb.js",
    "db:seed": "node src/seedBusinessData.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

3.5 public/app.js

文件路径：public/app.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const i18n = {
  zh: {
    systemTag: 'V1.0 留学生信息管理系统',
    systemName: '面向国际留学生的信息管理系统',
    logout: '退出',
    loginTitle: '用户登录',
    loginHint: '学生、教师和管理员使用统一入口登录。',
    username: '账号',
    password: '密码',
    loginButton: '登录系统',
    demoUsers: '演示账号',
    profile: '个人档案',
    grades: '成绩适配',
    learning: '学习情况',
    teacher: '教师管理',
```

```
    students: '学生管理',
    dashboard: '统计分析',
    risks: '风险查询',
    source: '来源',
    status: '状态',
    adaptedScore: '适配成绩',
    adaptedGpa: '适配 GPA',
    auditStatus: '审核状态',
    noData: '暂无数据',
    studentCount: '学生人数',
    teacherCount: '教师人数',
    visaWarnings: '签证预警',
    gradeExceptions: '成绩异常',
    loginFailed: '登录失败，请检查账号或密码。',
    loadFailed: '数据加载失败，请确认后台和数据库已启动。',
    studentOnly: '学生端支持中英文切换并仅展示本人数据。',
    teacherScope: '教师端用于班级、课程、成绩录入与学习预警管理。',
    adminScope: '管理员端用于档案、国籍、班级、住宿、签证、教师、成绩规则和统计分析管理。'
  },
  en: {
    systemTag: 'V1.0 International Student Information System',
    systemName: 'Information Management System for International Students',
    logout: 'Logout',
    loginTitle: 'Sign In',
    loginHint: 'Students, teachers, and administrators use the same portal.',
    username: 'Username',
    password: 'Password',
    loginButton: 'Sign In',
    demoUsers: 'Demo accounts',
    profile: 'Profile',
    grades: 'Grade Adapter',
    learning: 'Learning',
    teacher: 'Teacher Management',
    students: 'Student Management',
    dashboard: 'Analytics',
    risks: 'Risk Query',
    source: 'Source',
    status: 'Status',
    adaptedScore: 'Adapted Score',
    adaptedGpa: 'Adapted GPA',
    auditStatus: 'Audit Status',
    noData: 'No data',
    studentCount: 'Students',
    teacherCount: 'Teachers',
    visaWarnings: 'Visa Alerts',
    gradeExceptions: 'Grade Exceptions',
    loginFailed: 'Sign-in failed. Check your username or password.',
    loadFailed: 'Failed to load data. Confirm the API and database are running.',
    studentOnly: 'The student client supports Chinese and English and only shows personal data.',
    teacherScope: 'The teacher portal supports classes, courses, grade entry, and learning alerts.',
    adminScope: 'The admin portal manages profiles, nationalities, classes, dormitories, visas, teachers,
grade rules, and analytics.'
  }
};

let lang = localStorage.getItem('lang') || 'zh';
let session = JSON.parse(localStorage.getItem('session') || 'null');
let currentTab = 'profile';

const loginPanel = document.querySelector('#loginPanel');
const workspace = document.querySelector('#workspace');
const tabs = document.querySelector('#tabs');
const summaryGrid = document.querySelector('#summaryGrid');
const primaryView = document.querySelector('#primaryView');
const sideView = document.querySelector('#sideView');
const loginMessage = document.querySelector('#loginMessage');
const logoutBtn = document.querySelector('#logoutBtn');

function t(key) {
  return i18n[lang][key] || key;
```

```
}

function applyLanguage() {
  document.documentElement.lang = lang === 'zh' ? 'zh-CN' : 'en';
  document.querySelectorAll('[data-i18n]').forEach((node) => {
    node.textContent = t(node.dataset.i18n);
  });
  if (session) renderApp();
}

async function api(path, options = {}) {
  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };
  if (session?.token) headers.Authorization = `Bearer ${session.token}`;
  const response = await fetch(path, { ...options, headers });
  if (!response.ok) {
    const body = await response.json().catch(() => ({}));
    throw new Error(body.message || response.statusText);
  }
  return response.json();
}

function setLoggedIn(value) {
  loginPanel.classList.toggle('hidden', value);
  workspace.classList.toggle('hidden', !value);
  logoutBtn.classList.toggle('hidden', !value);
}

function roleTabs(role) {
  if (role === 'student') return ['profile', 'grades', 'learning'];
  if (role === 'teacher') return ['dashboard', 'students', 'grades', 'risks'];
  return ['dashboard', 'students', 'teacher', 'grades', 'risks'];
}

function renderTabs() {
  tabs.innerHTML = roleTabs(session.user.role)
    .map((tab) => `<button class="tab-button ${tab === currentTab ? 'active' : ''}" data-
tab="${tab}">${t(tab)}</button>`)
    .join('');
  tabs.querySelectorAll('button').forEach((button) => {
    button.addEventListener('click', () => {
      currentTab = button.dataset.tab;
      renderApp();
    });
  });
}

function metric(label, value, tone = '') {
  return `<div class="metric"><span class="muted">${label}</span><strong class="${tone}">${value}</strong>
</div>`;
}

function table(headers, rows) {
  if (!rows.length) return `<p class="muted">${t('noData')}</p>`;
  return `
    <table class="data-table">
      <thead><tr>${headers.map((item) => `<th>${item}</th>`).join('')}</tr></thead>
      <tbody>${rows.join('')}</tbody>
    </table>
  `;
}

async function renderDashboard() {
  const data = await api('/api/management/dashboard');
  summaryGrid.innerHTML = [
    metric(t('studentCount'), data.students),
    metric(t('teacherCount'), data.teachers),
    metric(t('visaWarnings'), data.visaWarnings, 'warning'),
```

```
    metric(t('gradeExceptions'), data.gradeExceptions, 'danger')
  ].join('');
  primaryView.innerHTML = `<h2>${t('dashboard')}</h2><p class="muted">${session.user.role === 'admin' ?
t('adminScope') : t('teacherScope')}</p>`;
  sideView.innerHTML = `<h3>${t('status')}</h3><p class="status">${session.user.displayName}</p>`;
}

async function renderStudentProfile() {
  const me = await api('/api/students/me');
  summaryGrid.innerHTML = [
    metric(t('profile'), me?.student_no || '-'),
    metric(lang === 'zh' ? '国籍' : 'Nationality', lang === 'zh' ? me?.nationality_zh || '-' :
me?.nationality_en || '-'),
    metric(lang === 'zh' ? '班级' : 'Class', me?.class_name || '-'),
    metric(t('status'), me?.status || '-')
  ].join('');
  primaryView.innerHTML = `
    <h2>${t('profile')}</h2>
    <div class="profile-list">
      <div><span>${lang === 'zh' ? '中文名' : 'Chinese Name'}</span>${me?.chinese_name || '-'}</div>
      <div><span>${lang === 'zh' ? 'English Name' : '英文名'}</span>${me?.english_name || '-'}</div>
      <div><span>${lang === 'zh' ? '护照号' : 'Passport No.'}</span>${me?.passport_no || '-'}</div>
      <div><span>${lang === 'zh' ? '护照有效期' : 'Passport Expiry'}</span>${me?.passport_expiry || '-'}
</div>
      <div><span>${lang === 'zh' ? '专业' : 'Major'}</span>${me?.major || '-'}</div>
      <div><span>${lang === 'zh' ? '邮箱' : 'Email'}</span>${me?.email || '-'}</div>
    </div>
  `;
  sideView.innerHTML = `<h3>${t('learning')}</h3><p class="muted">${t('studentOnly')}</p>`;
}

async function renderStudentGrades(studentId) {
  const id = studentId || (await api('/api/students/me'))?.id;
  const grades = id ? await api(`/api/grades/student/${id}`) : [];
  summaryGrid.innerHTML = [
    metric(t('grades'), grades.length),
    metric(t('adaptedGpa'), grades[0]?.adapted_gpa || '-'),
    metric(t('auditStatus'), grades[0]?.audit_status || '-'),
    metric(t('status'), grades[0]?.adapt_status || '-')
  ].join('');
  primaryView.innerHTML = `
    <h2>${t('grades')}</h2>
    ${table(
      [lang === 'zh' ? '课程' : 'Course', t('source'), lang === 'zh' ? '原始成绩' : 'Original',
t('adaptedScore'), t('adaptedGpa'), t('auditStatus')],
      grades.map(
        (item) => `<tr>
          <td>${item.course_name || '-'}</td>
          <td>${item.source_school_name || item.source_country_code || '-'}</td>
          <td>${item.original_grade || '-'}</td>
          <td>${item.adapted_score || '-'}</td>
          <td>${item.adapted_gpa || '-'}</td>
          <td><span class="status">${item.audit_status}</span></td>
        </tr>`
      )
    )}
  `;
  sideView.innerHTML = `<h3>${lang === 'zh' ? '成绩适配接口要点' : 'Adapter Notes'}</h3><p
class="muted">${lang === 'zh' ? '系统保留原始成绩、适配成绩、规则版本、审核状态和异常原因，支持跨国家、跨学校成绩归一化。'
: 'The system keeps original grades, adapted results, rule versions, audit states, and exception reasons
for cross-country grade normalization.'}</p>`;
}

async function renderGradeManagement() {
  const grades = await api('/api/grades');
  summaryGrid.innerHTML = [
    metric(t('grades'), grades.length),
    metric(t('adaptedGpa'), grades[0]?.adapted_gpa || '-'),
    metric(t('auditStatus'), grades[0]?.audit_status || '-'),
    metric(lang === 'zh' ? '异常记录' : 'Exceptions', grades.filter((item) => item.adapt_status ===
```

```
'exception').length, 'danger')
  ].join('');
  primaryView.innerHTML = `
    <h2>${t('grades')}</h2>
    ${table(
      [lang === 'zh' ? '学号' : 'Student No.', lang === 'zh' ? '学生' : 'Student', lang === 'zh' ? '课程' :
'Course', t('source'), lang === 'zh' ? '原始成绩' : 'Original', t('adaptedScore'), t('auditStatus')],
      grades.map(
        (item) => `<tr>
          <td>${item.student_no}</td>
          <td>${item.chinese_name || item.english_name}</td>
          <td>${item.course_name || '-'}</td>
          <td>${item.source_school_name || item.source_country_code || '-'}</td>
          <td>${item.original_grade || '-'}</td>
          <td>${item.adapted_score || '-'}</td>
          <td><span class="status ${item.adapt_status === 'exception' ? 'danger' :
''}">${item.audit_status}</span></td>
        </tr>`
      )
    )}
  `;
  sideView.innerHTML = `<h3>${lang === 'zh' ? '成绩适配管理' : 'Grade Adaptation'}</h3><p
class="muted">${lang === 'zh' ? '教师和管理员可查看外部学校成绩、原始成绩类型、适配分数、GPA、审核状态和异常记录，支撑跨国
家、跨学校成绩统一归档。' : 'Teachers and administrators can review external school grades, original types,
adapted scores, GPA, audit states, and exception records for unified archiving.'}</p>`;
}

async function renderStudents() {
  const students = await api('/api/students');
  summaryGrid.innerHTML = [metric(t('studentCount'), students.length), metric(t('status'),
session.user.role), metric(t('grades'), '-'), metric(t('visaWarnings'), '-')].join('');
  primaryView.innerHTML = `
    <h2>${t('students')}</h2>
    ${table(
      [lang === 'zh' ? '学号' : 'Student No.', lang === 'zh' ? '姓名' : 'Name', lang === 'zh' ? '国籍' :
'Nationality', lang === 'zh' ? '班级' : 'Class', t('status')],
      students.map(
        (item) => `<tr>
          <td>${item.student_no}</td>
          <td>${item.chinese_name || item.english_name}</td>
          <td>${item.nationality || '-'}</td>
          <td>${item.class_name || '-'}</td>
          <td><span class="status">${item.status}</span></td>
        </tr>`
      )
    )}
  `;
  sideView.innerHTML = `<h3>${t('teacher')}</h3><p class="muted">${session.user.role === 'teacher' ?
t('teacherScope') : t('adminScope')}</p>`;
}

async function renderTeachers() {
  const teachers = await api('/api/teachers');
  summaryGrid.innerHTML = [metric(t('teacherCount'), teachers.length), metric(t('status'), 'active'),
metric(lang === 'zh' ? '权限' : 'Role', 'RBAC'), metric(lang === 'zh' ? '日志' : 'Logs', 'on')].join('');
  primaryView.innerHTML = `
    <h2>${t('teacher')}</h2>
    ${table(
      [lang === 'zh' ? '工号' : 'Teacher No.', lang === 'zh' ? '姓名' : 'Name', lang === 'zh' ? '学院' :
'College', lang === 'zh' ? '类型' : 'Type', lang === 'zh' ? '邮箱' : 'Email'],
      teachers.map(
        (item) => `<tr>
          <td>${item.teacher_no}</td>
          <td>${item.name}</td>
          <td>${item.college || '-'}</td>
          <td>${item.teacher_type}</td>
          <td>${item.email || '-'}</td>
        </tr>`
      )
    )}
```

```
  `;
  sideView.innerHTML = `<h3>${lang === 'zh' ? '教师端权限' : 'Teacher Permissions'}</h3><p
class="muted">${t('teacherScope')}</p>`;
}

async function renderRisks() {
  const data = await api('/api/management/risks');
  const summary = data.summary || {};
  summaryGrid.innerHTML = [
    metric(lang === 'zh' ? '签证临期' : 'Visa Expiring', summary.visa_expiring || 0, 'warning'),
    metric(lang === 'zh' ? '成绩异常' : 'Grade Exceptions', summary.grade_exception || 0, 'danger'),
    metric(lang === 'zh' ? '低分风险' : 'Low Scores', summary.low_score || 0, 'danger'),
    metric(lang === 'zh' ? '学习预警' : 'Learning Alerts', summary.learning_warning || 0, 'warning')
  ].join('');
  primaryView.innerHTML = `
    <h2>${t('risks')}</h2>
    ${table(
      [lang === 'zh' ? '类型' : 'Type', lang === 'zh' ? '学号' : 'Student No.', lang === 'zh' ? '姓名' :
'Name', lang === 'zh' ? '日期' : 'Date', lang === 'zh' ? '详情' : 'Detail'],
      (data.items || []).map(
        (item) => `<tr>
          <td><span class="status ${item.risk_type.includes('exception') || item.risk_type === 'low_score'
? 'danger' : 'warning'}">${item.risk_type}</span></td>
          <td>${item.student_no}</td>
          <td>${item.chinese_name || item.english_name}</td>
          <td>${String(item.risk_date || '-').slice(0, 10)}</td>
          <td>${item.detail || '-'}</td>
        </tr>`
      )
    )}
  `;
  sideView.innerHTML = `<h3>${t('risks')}</h3><p class="muted">${lang === 'zh' ? '风险查询聚合签证、成绩、学习和
住宿等关键业务风险，便于教师和管理员及时处理。' : 'Risk query aggregates visa, grade, learning, and dormitory risks
for teachers and administrators.'}</p>`;
}

function renderLearningPlaceholder() {
  summaryGrid.innerHTML = [metric(t('learning'), '-'), metric(lang === 'zh' ? '考勤' : 'Attendance', '-'),
metric(lang === 'zh' ? '预警' : 'Alerts', '-'), metric(lang === 'zh' ? '课程' : 'Courses', '-')].join('');
  primaryView.innerHTML = `<h2>${t('learning')}</h2><p class="muted">${lang === 'zh' ? '学习情况模块用于展示课
程、考勤、作业、课堂表现、学习预警和教师评价。' : 'The learning module shows courses, attendance, assignments,
classroom performance, alerts, and teacher evaluations.'}</p>`;
  sideView.innerHTML = `<h3>${t('status')}</h3><p class="status">${lang === 'zh' ? '待接入数据' : 'Awaiting
records'}</p>`;
}

async function renderApp() {
  if (!session) return;
  setLoggedIn(true);
  renderTabs();
  try {
    if (currentTab === 'dashboard') await renderDashboard();
    if (currentTab === 'profile') await renderStudentProfile();
    if (currentTab === 'grades') {
      if (session.user.role === 'student') await renderStudentGrades();
      else await renderGradeManagement();
    }
    if (currentTab === 'students') await renderStudents();
    if (currentTab === 'teacher') await renderTeachers();
    if (currentTab === 'risks') await renderRisks();
    if (currentTab === 'learning') renderLearningPlaceholder();
  } catch (error) {
    primaryView.innerHTML = `<p class="message">${t('loadFailed')}</p><code>${error.message}</code>`;
    sideView.innerHTML = '';
  }
}

document.querySelector('#langToggle').addEventListener('click', () => {
  lang = lang === 'zh' ? 'en' : 'zh';
  localStorage.setItem('lang', lang);
```

```
  applyLanguage();
});

document.querySelector('#loginForm').addEventListener('submit', async (event) => {
  event.preventDefault();
  loginMessage.textContent = '';
  const form = new FormData(event.currentTarget);
  try {
    session = await api('/api/auth/login', {
      method: 'POST',
      body: JSON.stringify({
        username: form.get('username'),
        password: form.get('password')
      })
    });
    localStorage.setItem('session', JSON.stringify(session));
    currentTab = roleTabs(session.user.role)[0];
    renderApp();
  } catch (error) {
    loginMessage.textContent = t('loginFailed');
  }
});

logoutBtn.addEventListener('click', () => {
  session = null;
  localStorage.removeItem('session');
  setLoggedIn(false);
});

applyLanguage();
if (session) {
  currentTab = roleTabs(session.user.role)[0];
  renderApp();
} else {
  setLoggedIn(false);
}
```

3.6 public/index.html

文件路径：public/index.html

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>International Student Information Platform</title>
    <link rel="icon" href="/favicon.svg" type="image/svg+xml" />
    <link rel="stylesheet" href="/styles.css" />
  </head>
  <body>
    <header class="topbar">
      <div>
        <p class="eyebrow" data-i18n="systemTag">V1.0 留学生信息管理系统</p>
        <h1 data-i18n="systemName">面向国际留学生的信息管理系统</h1>
      </div>
      <div class="toolbar">
        <button class="icon-button" id="langToggle" title="Language">中 / EN</button>
        <button class="ghost-button hidden" id="logoutBtn" data-i18n="logout">退出</button>
      </div>
    </header>

    <main class="layout">
      <section class="login-panel" id="loginPanel">
        <div class="panel-title">
          <h2 data-i18n="loginTitle">用户登录</h2>
          <p data-i18n="loginHint">学生、教师和管理员使用统一入口登录。</p>
```

```
        </div>
        <form id="loginForm" class="form-grid">
          <label>
            <span data-i18n="username">账号</span>
            <input name="username" autocomplete="username" value="student01" required />
          </label>
          <label>
            <span data-i18n="password">密码</span>
            <input name="password" type="password" autocomplete="current-password" value="***" required />
          </label>
          <button class="primary-button" type="submit" data-i18n="loginButton">登录系统</button>
          <p class="message" id="loginMessage"></p>
        </form>
        <div class="demo-users">
          <span data-i18n="demoUsers">演示账号</span>
          <code>student01 / ******</code>
          <code>teacher01 / ******</code>
          <code>student001 / ******</code>
          <code>admin / ******</code>
        </div>
      </section>

      <section class="workspace hidden" id="workspace">
        <nav class="tabs" id="tabs"></nav>

        <section class="summary-grid" id="summaryGrid"></section>

        <section class="content-grid">
          <article class="surface" id="primaryView"></article>
          <aside class="surface compact" id="sideView"></aside>
        </section>
      </section>
    </main>

    <script src="/app.js"></script>
  </body>
</html>
```

3.7 public/styles.css

文件路径：public/styles.css

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
:root {
  color-scheme: light;
  --bg: #f6f7f9;
  --surface: #ffffff;
  --surface-strong: #eef4f1;
  --line: #d8dee6;
  --text: #1f2933;
  --muted: #64707d;
  --accent: #1f7a6d;
  --accent-dark: #14594f;
  --warning: #b45f06;
  --danger: #b42318;
  --blue: #2f5f98;
  --shadow: 0 14px 34px rgba(21, 39, 60, 0.08);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background: var(--bg);
  color: var(--text);
```

```
  font-family: Arial, "Microsoft YaHei", sans-serif;
}

button,
input,
select {
  font: inherit;
}

.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  padding: 24px clamp(16px, 4vw, 48px);
  background: #ffffff;
  border-bottom: 1px solid var(--line);
}

.eyebrow {
  margin: 0 0 6px;
  color: var(--accent);
  font-size: 13px;
  font-weight: 700;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 0;
  font-size: clamp(22px, 3vw, 34px);
}

h2 {
  margin-bottom: 8px;
  font-size: 22px;
}

h3 {
  margin-bottom: 12px;
  font-size: 17px;
}

.toolbar,
.tabs,
.actions {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.layout {
  width: min(1180px, calc(100% - 32px));
  margin: 28px auto 48px;
}

.login-panel,
.surface {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 8px;
  box-shadow: var(--shadow);
}
```

```
.login-panel {
  width: min(520px, 100%);
  padding: 26px;
}

.panel-title p,
.muted {
  color: var(--muted);
}

.form-grid {
  display: grid;
  gap: 16px;
}

label {
  display: grid;
  gap: 6px;
  color: var(--muted);
  font-size: 14px;
}

input,
select {
  width: 100%;
  min-height: 42px;
  padding: 9px 11px;
  border: 1px solid var(--line);
  border-radius: 6px;
  background: #ffffff;
  color: var(--text);
}

.primary-button,
.ghost-button,
.icon-button,
.tab-button {
  min-height: 40px;
  border-radius: 6px;
  border: 1px solid transparent;
  cursor: pointer;
}

.primary-button {
  background: var(--accent);
  color: #ffffff;
  font-weight: 700;
}

.primary-button:hover {
  background: var(--accent-dark);
}

.ghost-button,
.icon-button,
.tab-button {
  background: #ffffff;
  border-color: var(--line);
  color: var(--text);
}

.icon-button {
  min-width: 76px;
}

.tab-button.active {
  background: var(--surface-strong);
  border-color: var(--accent);
  color: var(--accent-dark);
  font-weight: 700;
```

```
}

.message {
  min-height: 20px;
  margin-bottom: 0;
  color: var(--danger);
}

.demo-users {
  display: grid;
  gap: 8px;
  margin-top: 18px;
  color: var(--muted);
  font-size: 13px;
}

code {
  display: inline-block;
  width: fit-content;
  padding: 4px 7px;
  background: #eef1f4;
  border-radius: 5px;
  color: #27313d;
}

.workspace {
  display: grid;
  gap: 18px;
}

.summary-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
}

.metric {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 18px;
}

.metric strong {
  display: block;
  margin-top: 8px;
  font-size: 28px;
}

.content-grid {
  display: grid;
  grid-template-columns: minmax(0, 2fr) minmax(280px, 0.9fr);
  gap: 18px;
}

.surface {
  padding: 20px;
}

.compact {
  align-self: start;
}

.data-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 14px;
}

.data-table th,
```

```
.data-table td {
  padding: 11px 9px;
  border-bottom: 1px solid var(--line);
  text-align: left;
  vertical-align: top;
}

.data-table th {
  color: var(--muted);
  font-size: 13px;
}

.status {
  display: inline-flex;
  align-items: center;
  min-height: 24px;
  padding: 2px 8px;
  border-radius: 999px;
  background: #e7f3ef;
  color: var(--accent-dark);
  font-size: 12px;
  font-weight: 700;
}

.status.warning {
  background: #fff4df;
  color: var(--warning);
}

.status.danger {
  background: #fee4e2;
  color: var(--danger);
}

.profile-list {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}

.profile-list div {
  padding: 12px;
  background: #f9fafb;
  border: 1px solid var(--line);
  border-radius: 6px;
}

.profile-list span {
  display: block;
  margin-bottom: 4px;
  color: var(--muted);
  font-size: 12px;
}

.hidden {
  display: none !important;
}

@media (max-width: 820px) {
  .topbar {
    align-items: flex-start;
    flex-direction: column;
  }

  .summary-grid,
  .content-grid,
  .profile-list {
    grid-template-columns: 1fr;
  }
```

```
  .login-panel {
    padding: 20px;
  }
}
```

3.8 src/config/db.js

文件路径：src/config/db.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST || '***',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '***',
  database: process.env.DB_NAME || 'international_student_platform',
  waitForConnections: true,
  connectionLimit: 10,
  namedPlaceholders: true
});

module.exports = pool;
```

3.9 src/initDb.js

文件路径：src/initDb.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const fs = require('fs');
const path = require('path');
const mysql = require('mysql2/promise');
require('dotenv').config();

async function initDb() {
  const sql = fs.readFileSync(path.join(__dirname, '..', 'database', 'schema.sql'), 'utf8');
  const connection = await mysql.createConnection({
    host: process.env.DB_HOST || '***',
    port: Number(process.env.DB_PORT || 3306),
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '***',
    multipleStatements: true
  });

  await connection.query(sql);
  await connection.end();
  console.log('Database initialized.');
}

initDb().catch((error) => {
  console.error(error.message);
  process.exit(1);
});
```

3.10 src/middleware/auth.js

文件路径：src/middleware/auth.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const jwt = require('jsonwebtoken');

function auth(requiredRoles = []) {
  return (req, res, next) => {
    const header = req.headers.authorization || '';
```

```
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';

    if (!token) {
      return res.status(401).json({ message: 'Missing authorization token' });
    }

    try {
      const payload = jwt.verify(token, process.env.JWT_SECRET || '***');
      if (requiredRoles.length && !requiredRoles.includes(payload.role)) {
        return res.status(403).json({ message: 'Permission denied' });
      }
      req.user = payload;
      next();
    } catch (error) {
      return res.status(401).json({ message: 'Invalid or expired token' });
    }
  };
}

module.exports = auth;
```

3.11 src/routes/auth.js

文件路径：src/routes/auth.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../config/db');

const router = express.Router();

router.post('/login', async (req, res, next) => {
  try {
    const { username, password } = req.body;
    const [rows] = await pool.execute(
      'SELECT id, username, password_hash, role, display_name, status FROM users WHERE username = :username
LIMIT 1',
      { username }
    );

    const user = rows[0];
    if (!user || user.status !== 'active') {
      return res.status(401).json({ message: 'Invalid username or password' });
    }

    const ok = await bcrypt.compare(password, user.password_hash);
    if (!ok) {
      return res.status(401).json({ message: 'Invalid username or password' });
    }

    const token = jwt.sign(
      { id: user.id, username: user.username, role: user.role, name: user.display_name },
      process.env.JWT_SECRET || '***',
      { expiresIn: process.env.JWT_EXPIRES_IN || '8h' }
    );

    res.json({
      token,
      user: {
        id: user.id,
        username: user.username,
        role: user.role,
        displayName: user.display_name
      }
    });
  } catch (error) {
```

```
    next(error);
  }
});

module.exports = router;
```

3.12 src/routes/grades.js

文件路径：src/routes/grades.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');
const { adaptGrade } = require('../services/gradeAdapter');

const router = express.Router();

async function findRule(schoolId, countryCode) {
  const [rows] = await pool.execute(
    `SELECT *
     FROM grade_rules
     WHERE (school_id = :schoolId OR school_id IS NULL)
       AND (country_code = :countryCode OR country_code IS NULL)
       AND status = 'active'
     ORDER BY school_id DESC, id DESC
     LIMIT 1`,
    { schoolId: schoolId || null, countryCode: countryCode || null }
  );
  const rule = rows[0];
  if (!rule) return {};

  return {
    gpaScale: rule.gpa_scale,
    creditRatio: rule.credit_ratio,
    passScore: rule.pass_score,
    version: rule.version
  };
}

router.get('/', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT gr.*, s.student_no, s.english_name, s.chinese_name, es.name AS source_school_name
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       LEFT JOIN external_schools es ON es.id = gr.source_school_id
       ORDER BY gr.id DESC
       LIMIT 200`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/adapt', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const input = req.body;
    const rule = await findRule(input.schoolId, input.countryCode);
    const adapted = adaptGrade(input, rule);

    const [result] = await pool.execute(
      `INSERT INTO grade_records
       (student_id, course_code, course_name, source_school_id, source_country_code,
        original_grade, original_type, original_credit, adapted_score, adapted_gpa,
        adapted_credit, passed, rule_version, adapt_status, audit_status)
       VALUES (:studentId, :courseCode, :courseName, :schoolId, :countryCode,

```

```
        :originalGrade, :originalType, :originalCredit, :adaptedScore, :adaptedGpa,
        :adaptedCredit, :passed, :ruleVersion, 'success', 'pending')`,
      {
        studentId: input.studentId,
        courseCode: input.courseCode,
        courseName: input.courseName,
        schoolId: input.schoolId || null,
        countryCode: input.countryCode || null,
        originalGrade: String(input.originalGrade),
        originalType: input.originalType,
        originalCredit: input.originalCredit || 0,
        adaptedScore: adapted.adaptedScore,
        adaptedGpa: adapted.adaptedGpa,
        adaptedCredit: adapted.adaptedCredit,
        passed: adapted.passed ? 1 : 0,
        ruleVersion: adapted.ruleVersion
      }
    );

    res.status(201).json({ id: result.insertId, ...adapted, auditStatus: 'pending' });
  } catch (error) {
    if (req.body && req.body.studentId) {
      await pool.execute(
        `INSERT INTO grade_records
         (student_id, course_code, course_name, source_school_id, source_country_code,
          original_grade, original_type, original_credit, adapt_status, exception_reason, audit_status)
         VALUES (:studentId, :courseCode, :courseName, :schoolId, :countryCode,
          :originalGrade, :originalType, :originalCredit, 'exception', :reason, 'pending')`,
        {
          studentId: req.body.studentId,
          courseCode: req.body.courseCode || null,
          courseName: req.body.courseName || null,
          schoolId: req.body.schoolId || null,
          countryCode: req.body.countryCode || null,
          originalGrade: String(req.body.originalGrade || ''),
          originalType: req.body.originalType || null,
          originalCredit: req.body.originalCredit || 0,
          reason: error.message
        }
      );
    }
    res.status(422).json({ message: error.message });
  }
});

router.get('/student/:studentId', auth(['admin', 'teacher', 'student']), async (req, res, next) => {
  try {
    if (req.user.role === 'student') {
      const [owned] = await pool.execute('SELECT id FROM students WHERE user_id = :userId', { userId:
req.user.id });
      if (!owned[0] || String(owned[0].id) !== String(req.params.studentId)) {
        return res.status(403).json({ message: 'Permission denied' });
      }
    }

    const [rows] = await pool.execute(
      `SELECT gr.*, es.name AS source_school_name
       FROM grade_records gr
       LEFT JOIN external_schools es ON es.id = gr.source_school_id
       WHERE gr.student_id = :studentId
       ORDER BY gr.id DESC`,
      { studentId: req.params.studentId }
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.patch('/:id/audit', auth(['admin']), async (req, res, next) => {
```

```
  try {
    const { auditStatus } = req.body;
    await pool.execute(
      'UPDATE grade_records SET audit_status = :auditStatus, audited_by = :auditedBy, audited_at = NOW()
WHERE id = :id',
      { auditStatus, auditedBy: req.user.id, id: req.params.id }
    );
    res.json({ id: Number(req.params.id), auditStatus });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

3.13 src/routes/management.js

文件路径：src/routes/management.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/dashboard', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [[students]] = await pool.execute('SELECT COUNT(*) AS count FROM students');
    const [[teachers]] = await pool.execute('SELECT COUNT(*) AS count FROM teachers');
    const [[visaWarnings]] = await pool.execute(
      'SELECT COUNT(*) AS count FROM visas WHERE expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
INTERVAL 60 DAY)'
    );
    const [[gradeExceptions]] = await pool.execute(
      "SELECT COUNT(*) AS count FROM grade_records WHERE adapt_status = 'exception'"
    );

    res.json({
      students: students.count,
      teachers: teachers.count,
      visaWarnings: visaWarnings.count,
      gradeExceptions: gradeExceptions.count
    });
  } catch (error) {
    next(error);
  }
});

router.get('/nationalities', auth(['admin', 'teacher', 'student']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute('SELECT * FROM nationalities ORDER BY name_en');
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.get('/classes', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT c.*, t.name AS teacher_name
       FROM classes c
       LEFT JOIN teachers t ON t.id = c.teacher_id
       ORDER BY c.id DESC`
    );
    res.json(rows);
  } catch (error) {
```

```
    next(error);
  }
});

router.get('/risks', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [visaRisks] = await pool.execute(
      `SELECT 'visa_expiring' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              v.expiry_date AS risk_date,
              CONCAT('Visa expires on ', DATE_FORMAT(v.expiry_date, '%Y-%m-%d')) AS detail
       FROM visas v
       JOIN students s ON s.id = v.student_id
       WHERE v.expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 60 DAY)
       ORDER BY v.expiry_date ASC
       LIMIT 100`
    );

    const [gradeRisks] = await pool.execute(
      `SELECT 'grade_exception' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              gr.created_at AS risk_date,
              CONCAT(COALESCE(gr.course_name, 'Unknown course'), ': ', COALESCE(gr.exception_reason,
gr.adapt_status)) AS detail
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       WHERE gr.adapt_status = 'exception'
       ORDER BY gr.created_at DESC
       LIMIT 100`
    );

    const [lowScoreRisks] = await pool.execute(
      `SELECT 'low_score' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              gr.created_at AS risk_date,
              CONCAT(COALESCE(gr.course_name, 'Unknown course'), ': adapted score ', gr.adapted_score) AS
detail
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       WHERE gr.adapted_score IS NOT NULL AND gr.adapted_score < 60
       ORDER BY gr.adapted_score ASC
       LIMIT 100`
    );

    const [learningRisks] = await pool.execute(
      `SELECT 'learning_warning' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              sr.recorded_at AS risk_date,
              CONCAT(sr.course_name, ': ', sr.warning_level, ', attendance ', COALESCE(sr.attendance_rate,
0), '%') AS detail
       FROM study_records sr
       JOIN students s ON s.id = sr.student_id
       WHERE sr.warning_level IN ('warning', 'serious') OR sr.attendance_rate < 70
       ORDER BY sr.recorded_at DESC
       LIMIT 100`
    );

    const [dormitoryRisks] = await pool.execute(
      `SELECT 'dormitory_exception' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              d.check_out_date AS risk_date,
              CONCAT(d.building, '-', d.room_no, '-', d.bed_no, ': ', d.status) AS detail
       FROM dormitories d
       JOIN students s ON s.id = d.student_id
       WHERE d.status IN ('maintenance', 'reserved') OR d.check_out_date < CURDATE()
       ORDER BY d.check_out_date ASC
       LIMIT 100`
    );

    const items = [...visaRisks, ...gradeRisks, ...lowScoreRisks, ...learningRisks, ...dormitoryRisks];
    const summary = items.reduce((acc, item) => {
      acc[item.risk_type] = (acc[item.risk_type] || 0) + 1;
      return acc;
    }, {});
```

```
    res.json({ summary, items });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

3.14 src/routes/students.js

文件路径：src/routes/students.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/me', auth(['student']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT s.*, c.name AS class_name, n.name_zh AS nationality_zh, n.name_en AS nationality_en
       FROM students s
       LEFT JOIN classes c ON c.id = s.class_id
       LEFT JOIN nationalities n ON n.id = s.nationality_id
       WHERE s.user_id = :userId`,
      { userId: req.user.id }
    );
    res.json(rows[0] || null);
  } catch (error) {
    next(error);
  }
});

router.get('/', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT s.id, s.student_no, s.chinese_name, s.english_name, s.gender, s.enrollment_year,
              s.status, c.name AS class_name, n.name_zh AS nationality
       FROM students s
       LEFT JOIN classes c ON c.id = s.class_id
       LEFT JOIN nationalities n ON n.id = s.nationality_id
       ORDER BY s.id DESC`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/', auth(['admin']), async (req, res, next) => {
  try {
    const data = req.body;
    const [result] = await pool.execute(
      `INSERT INTO students
       (user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
        passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year,
        major, college, class_id, status)
       VALUES (:userId, :studentNo, :chineseName, :englishName, :gender, :birthDate, :nationalityId,
        :passportNo, :passportExpiry, :phone, :email, :emergencyContact, :enrollmentYear,
        :major, :college, :classId, :status)`,
      {
        userId: data.userId || null,
        studentNo: data.studentNo,
        chineseName: data.chineseName || null,
        englishName: data.englishName,
        gender: data.gender || null,
```

```
        birthDate: data.birthDate || null,
        nationalityId: data.nationalityId || null,
        passportNo: data.passportNo || null,
        passportExpiry: data.passportExpiry || null,
        phone: data.phone || null,
        email: data.email || null,
        emergencyContact: data.emergencyContact || null,
        enrollmentYear: data.enrollmentYear || null,
        major: data.major || null,
        college: data.college || null,
        classId: data.classId || null,
        status: data.status || 'active'
      }
    );
    res.status(201).json({ id: result.insertId });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

3.15 src/routes/teachers.js

文件路径：src/routes/teachers.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/', auth(['admin']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT t.*, u.username
       FROM teachers t
       LEFT JOIN users u ON u.id = t.user_id
       ORDER BY t.id DESC`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.get('/me/classes', auth(['teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT c.*
       FROM classes c
       JOIN teachers t ON t.id = c.teacher_id
       WHERE t.user_id = :userId`,
      { userId: req.user.id }
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/', auth(['admin']), async (req, res, next) => {
  try {
    const data = req.body;
    const [result] = await pool.execute(
      `INSERT INTO teachers (user_id, teacher_no, name, college, phone, email, teacher_type)
       VALUES (:userId, :teacherNo, :name, :college, :phone, :email, :teacherType)`,
```

```
      {
        userId: data.userId || null,
        teacherNo: data.teacherNo,
        name: data.name,
        college: data.college || null,
        phone: data.phone || null,
        email: data.email || null,
        teacherType: data.teacherType || 'course_teacher'
      }
    );
    res.status(201).json({ id: result.insertId });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

3.16 src/seedBusinessData.js

文件路径：src/seedBusinessData.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const bcrypt = require('bcryptjs');
const pool = require('./config/db');
const { adaptGrade } = require('./services/gradeAdapter');

const passwordHash = bcrypt.hashSync('***', 10);
const countryCodes = [
  'TH', 'VN', 'MY', 'ID', 'PH', 'SG', 'LA', 'KH', 'MM', 'BN',
  'CN', 'JP', 'KR', 'IN', 'PK', 'BD', 'LK', 'NP', 'MN', 'KZ'
];
const gradeTypes = ['percent', 'letter', 'gpa', 'pass_fail'];
const letters = ['A', 'A-', 'B', 'B-', 'C', 'C-', 'D', 'F'];

function pad(num, size = 3) {
  return String(num).padStart(size, '0');
}

function datePlus(days) {
  const date = new Date();
  date.setDate(date.getDate() + days);
  return date.toISOString().slice(0, 10);
}

async function execute(sql, params = {}) {
  await pool.execute(sql, params);
}

async function seedUsers() {
  await execute(
    `INSERT INTO users (id, username, password_hash, role, display_name, status)
     VALUES (1, 'admin', :passwordHash, 'admin', 'System Admin', 'active')
     ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), status = 'active'`,
    { passwordHash }
  );

  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO users (id, username, password_hash, role, display_name, status)
       VALUES (:id, :username, :passwordHash, :role, :displayName, 'active')
       ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), role = VALUES(role), display_name =
VALUES(display_name), status = 'active'`,
      {
        id: 1000 + i,
        username: `teacher${pad(i)}`,
        passwordHash,
        role: 'teacher',
```

```
        displayName: `Teacher ${pad(i)}`
      }
    );
  }

  for (let i = 1; i <= 180; i += 1) {
    await execute(
      `INSERT INTO users (id, username, password_hash, role, display_name, status)
       VALUES (:id, :username, :passwordHash, 'student', :displayName, 'active')
       ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), role = 'student', display_name =
VALUES(display_name), status = 'active'`,
      {
        id: 2000 + i,
        username: `student${pad(i)}`,
        passwordHash,
        displayName: `International Student ${pad(i)}`
      }
    );
  }
}

async function seedNationalities() {
  for (let i = 1; i <= 120; i += 1) {
    const code = `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i / countryCodes.length),
2)}`;
    await execute(
      `INSERT INTO nationalities (id, country_code, name_zh, name_en, continent, primary_language,
timezone)
       VALUES (:id, :countryCode, :nameZh, :nameEn, 'Asia-Pacific', :language, :timezone)
       ON DUPLICATE KEY UPDATE name_zh = VALUES(name_zh), name_en = VALUES(name_en), primary_language =
VALUES(primary_language), timezone = VALUES(timezone)`,
      {
        id: i,
        countryCode: code,
        nameZh: `国际生来源地${pad(i)}`,
        nameEn: `International Region ${pad(i)}`,
        language: `Language ${pad(((i - 1) % 20) + 1, 2)}`,
        timezone: `Asia/Test-${pad(((i - 1) % 24) + 1, 2)}`
      }
    );
  }
}

async function seedTeachers() {
  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
       VALUES (:id, :userId, :teacherNo, :name, :college, :phone, :email, :teacherType)
       ON DUPLICATE KEY UPDATE user_id = VALUES(user_id), name = VALUES(name), college = VALUES(college),
phone = VALUES(phone), email = VALUES(email), teacher_type = VALUES(teacher_type)`,
      {
        id: i,
        userId: 1000 + i,
        teacherNo: `T2026${pad(i)}`,
        name: `Teacher ${pad(i)}`,
        college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
        phone: `1380000${pad(i, 4)}`,
        email: `teacher${pad(i)}@example.edu`,
        teacherType: ['course_teacher', 'head_teacher', 'counselor', 'admin_teacher'][i % 4]
      }
    );
  }
}

async function seedClasses() {
  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
       VALUES (:id, :code, :name, :college, :major, :teacherId, :year)
       ON DUPLICATE KEY UPDATE name = VALUES(name), college = VALUES(college), major = VALUES(major),
```

```
teacher_id = VALUES(teacher_id), enrollment_year = VALUES(enrollment_year)`,
      {
        id: i,
        code: `ISE-2026-${pad(i)}`,
        name: `International Student Class ${pad(i)}`,
        college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
        major: ['Chinese Language', 'Business Chinese', 'Computer Science', 'International Trade'][i % 4],
        teacherId: ((i - 1) % 120) + 1,
        year: 2024 + (i % 3)
      }
    );
  }
}

async function seedStudents() {
  for (let i = 1; i <= 180; i += 1) {
    await execute(
      `INSERT INTO students
       (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
        passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, college,
class_id, status)
       VALUES (:id, :userId, :studentNo, :chineseName, :englishName, :gender, :birthDate, :nationalityId,
        :passportNo, :passportExpiry, :phone, :email, :emergencyContact, :enrollmentYear, :major, :college,
:classId, :status)
       ON DUPLICATE KEY UPDATE user_id = VALUES(user_id), chinese_name = VALUES(chinese_name), english_name
= VALUES(english_name),
        nationality_id = VALUES(nationality_id), class_id = VALUES(class_id), status = VALUES(status)`,
      {
        id: i,
        userId: 2000 + i,
        studentNo: `S2026${pad(i)}`,
        chineseName: `留学生${pad(i)}`,
        englishName: `International Student ${pad(i)}`,
        gender: i % 2 === 0 ? 'Male' : 'Female',
        birthDate: `200${i % 6}-0${(i % 9) + 1}-15`,
        nationalityId: ((i - 1) % 120) + 1,
        passportNo: `P${pad(i, 6)}`,
        passportExpiry: datePlus(365 + i),
        phone: `1390000${pad(i, 4)}`,
        email: `student${pad(i)}@example.com`,
        emergencyContact: `Family Contact ${pad(i)}`,
        enrollmentYear: 2024 + (i % 3),
        major: ['Chinese Language', 'Business Chinese', 'Computer Science', 'International Trade'][i % 4],
        college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
        classId: ((i - 1) % 120) + 1,
        status: ['active', 'active', 'active', 'delayed', 'suspended'][i % 5]
      }
    );
  }
}

async function seedDormitories() {
  for (let i = 1; i <= 180; i += 1) {
    await execute(
      `INSERT INTO dormitories (id, building, room_no, bed_no, student_id, check_in_date, check_out_date,
status)
       VALUES (:id, :building, :roomNo, :bedNo, :studentId, :checkIn, :checkOut, :status)
       ON DUPLICATE KEY UPDATE student_id = VALUES(student_id), check_out_date = VALUES(check_out_date),
status = VALUES(status)`,
      {
        id: i,
        building: `International Dorm ${((i - 1) % 8) + 1}`,
        roomNo: `${100 + ((i - 1) % 60)}`,
        bedNo: `${((i - 1) % 4) + 1}`,
        studentId: i,
        checkIn: '2026-09-01',
        checkOut: i % 17 === 0 ? datePlus(-5) : datePlus(200 + i),
        status: i % 29 === 0 ? 'maintenance' : 'occupied'
      }
    );
```

```
  }
}

async function seedVisas() {
  for (let i = 1; i <= 180; i += 1) {
    await execute(
      `INSERT INTO visas (id, student_id, visa_type, visa_no, issue_date, expiry_date, residence_permit_no,
document_url, status)
       VALUES (:id, :studentId, :visaType, :visaNo, :issueDate, :expiryDate, :permitNo, :documentUrl,
:status)
       ON DUPLICATE KEY UPDATE expiry_date = VALUES(expiry_date), status = VALUES(status)`,
      {
        id: i,
        studentId: i,
        visaType: i % 2 === 0 ? 'X1' : 'X2',
        visaNo: `VISA${pad(i, 6)}`,
        issueDate: '2026-01-10',
        expiryDate: i % 9 === 0 ? datePlus(20 + (i % 30)) : datePlus(180 + i),
        permitNo: `RP${pad(i, 6)}`,
        documentUrl: `/uploads/visa/${pad(i)}.pdf`,
        status: i % 9 === 0 ? 'expiring' : 'valid'
      }
    );
  }
}

async function seedStudyRecords() {
  for (let i = 1; i <= 240; i += 1) {
    const attendance = 55 + (i % 46);
    await execute(
      `INSERT INTO study_records
       (id, student_id, course_code, course_name, attendance_rate, homework_status, class_performance,
warning_level, teacher_id)
       VALUES (:id, :studentId, :courseCode, :courseName, :attendance, :homeworkStatus, :performance,
:warningLevel, :teacherId)
       ON DUPLICATE KEY UPDATE attendance_rate = VALUES(attendance_rate), warning_level =
VALUES(warning_level), teacher_id = VALUES(teacher_id)`,
      {
        id: i,
        studentId: ((i - 1) % 180) + 1,
        courseCode: `CN${100 + (i % 12)}`,
        courseName: ['Chinese Listening', 'Chinese Speaking', 'Chinese Reading', 'Academic Writing'][i %
4],
        attendance,
        homeworkStatus: attendance < 70 ? 'Incomplete' : 'Completed',
        performance: attendance < 70 ? 'Needs support' : 'Normal',
        warningLevel: attendance < 65 ? 'serious' : attendance < 75 ? 'warning' : 'none',
        teacherId: ((i - 1) % 120) + 1
      }
    );
  }
}

async function seedExternalSchools() {
  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO external_schools (id, country_code, name, contact_name, contact_email, integration_type,
endpoint_url, status)
       VALUES (:id, :countryCode, :name, :contactName, :contactEmail, :integrationType, :endpointUrl,
'active')
       ON DUPLICATE KEY UPDATE country_code = VALUES(country_code), name = VALUES(name), integration_type =
VALUES(integration_type), endpoint_url = VALUES(endpoint_url), status = 'active'`,
      {
        id: i,
        countryCode: `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i /
countryCodes.length), 2)}`,
        name: `Partner School ${pad(i)}`,
        contactName: `Partner Contact ${pad(i)}`,
        contactEmail: `partner${pad(i)}@school.example`,
        integrationType: ['api', 'excel', 'csv', 'manual', 'scheduled_sync'][i % 5],
```

```
        endpointUrl: `https://partner${pad(i)}.example.edu/api/grades`
      }
    );
  }
}

async function seedGradeRules() {
  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO grade_rules (id, school_id, country_code, name, version, gpa_scale, credit_ratio,
pass_score, status, effective_from, effective_to)
       VALUES (:id, :schoolId, :countryCode, :name, :version, :gpaScale, :creditRatio, :passScore,
'active', '2026-01-01', '2028-12-31')
       ON DUPLICATE KEY UPDATE gpa_scale = VALUES(gpa_scale), credit_ratio = VALUES(credit_ratio),
pass_score = VALUES(pass_score), status = 'active'`,
      {
        id: i,
        schoolId: i,
        countryCode: `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i /
countryCodes.length), 2)}`,
        name: `Grade Adaptation Rule ${pad(i)}`,
        version: `RULE-2026-${pad(i)}`,
        gpaScale: i % 3 === 0 ? 5 : 4,
        creditRatio: 0.8 + ((i % 5) * 0.1),
        passScore: i % 4 === 0 ? 65 : 60
      }
    );
  }
}

function originalGrade(i, type) {
  if (type === 'percent') return String(45 + (i % 56));
  if (type === 'letter') return letters[i % letters.length];
  if (type === 'gpa') return String(((i % 41) / 10).toFixed(1));
  return i % 6 === 0 ? 'fail' : 'pass';
}

async function seedGradeRecords() {
  for (let i = 1; i <= 360; i += 1) {
    const type = gradeTypes[i % gradeTypes.length];
    const studentId = ((i - 1) % 180) + 1;
    const schoolId = ((i - 1) % 120) + 1;
    const countryCode = `${countryCodes[(schoolId - 1) % countryCodes.length]}${pad(Math.ceil(schoolId /
countryCodes.length), 2)}`;
    const grade = originalGrade(i, type);
    const rule = {
      gpaScale: schoolId % 3 === 0 ? 5 : 4,
      creditRatio: 0.8 + ((schoolId % 5) * 0.1),
      passScore: schoolId % 4 === 0 ? 65 : 60,
      version: `RULE-2026-${pad(schoolId)}`
    };

    if (i % 37 === 0) {
      await execute(
        `INSERT INTO grade_records
         (id, student_id, course_code, course_name, source_school_id, source_country_code,
          original_grade, original_type, original_credit, adapt_status, exception_reason, audit_status)
         VALUES (:id, :studentId, :courseCode, :courseName, :schoolId, :countryCode,
          'N/A', 'letter', :credit, 'exception', 'Source grade cannot be mapped', 'pending')
         ON DUPLICATE KEY UPDATE adapt_status = 'exception', exception_reason = VALUES(exception_reason),
audit_status = 'pending'`,
        {
          id: i,
          studentId,
          courseCode: `CR${100 + (i % 20)}`,
          courseName: ['Mathematics', 'Chinese Language', 'Computer Basics', 'Economics'][i % 4],
          schoolId,
          countryCode,
          credit: 1 + (i % 4)
        }
```

```
      );
      continue;
    }

    const adapted = adaptGrade(
      { originalType: type, originalGrade: grade, originalCredit: 1 + (i % 4) },
      rule
    );

    await execute(
      `INSERT INTO grade_records
       (id, student_id, course_code, course_name, source_school_id, source_country_code,
        original_grade, original_type, original_credit, adapted_score, adapted_gpa, adapted_credit,
        passed, rule_version, adapt_status, audit_status, audited_by, audited_at)
       VALUES (:id, :studentId, :courseCode, :courseName, :schoolId, :countryCode,
        :originalGrade, :originalType, :originalCredit, :adaptedScore, :adaptedGpa, :adaptedCredit,
        :passed, :ruleVersion, 'success', :auditStatus, 1, NOW())
       ON DUPLICATE KEY UPDATE adapted_score = VALUES(adapted_score), adapted_gpa = VALUES(adapted_gpa),
        adapted_credit = VALUES(adapted_credit), passed = VALUES(passed), rule_version =
VALUES(rule_version),
        adapt_status = 'success', audit_status = VALUES(audit_status)`,
      {
        id: i,
        studentId,
        courseCode: `CR${100 + (i % 20)}`,
        courseName: ['Mathematics', 'Chinese Language', 'Computer Basics', 'Economics'][i % 4],
        schoolId,
        countryCode,
        originalGrade: grade,
        originalType: type,
        originalCredit: 1 + (i % 4),
        adaptedScore: adapted.adaptedScore,
        adaptedGpa: adapted.adaptedGpa,
        adaptedCredit: adapted.adaptedCredit,
        passed: adapted.passed ? 1 : 0,
        ruleVersion: adapted.ruleVersion,
        auditStatus: i % 5 === 0 ? 'pending' : 'approved'
      }
    );
  }
}

async function seedDemoAliases() {
  await execute(
    `INSERT INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
     VALUES (121, 2, 'T-DEMO-001', 'Teacher Demo', 'International Education College', '13800009999',
'***@***', 'head_teacher')
     ON DUPLICATE KEY UPDATE user_id = 2, name = VALUES(name), college = VALUES(college), phone =
VALUES(phone), email = VALUES(email), teacher_type = VALUES(teacher_type)`
  );

  await execute(
    `INSERT INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
     VALUES (121, 'ISE-DEMO-001', 'Demo International Student Class', 'International Education College',
'Chinese Language', 121, 2026)
     ON DUPLICATE KEY UPDATE name = VALUES(name), teacher_id = 121, enrollment_year = 2026`
  );

  await execute(
    `INSERT INTO students
     (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
      passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, college,
class_id, status)
     VALUES (181, 3, 'S-DEMO-001', '学生示例', 'Student Demo', 'Female', '2004-08-01', 1,
      'PDEMO001', :passportExpiry, '13900009999', '***@***', 'Family Demo', 2026,
      'Chinese Language', 'International Education College', 121, 'active')
     ON DUPLICATE KEY UPDATE user_id = 3, class_id = 121, status = 'active'`,
    { passportExpiry: datePlus(900) }
  );
```

```
  await execute(
    `INSERT INTO dormitories (id, building, room_no, bed_no, student_id, check_in_date, check_out_date,
status)
     VALUES (181, 'International Dorm Demo', 'D101', '1', 181, '2026-09-01', :checkOut, 'occupied')
     ON DUPLICATE KEY UPDATE student_id = 181, check_out_date = VALUES(check_out_date), status =
'occupied'`,
    { checkOut: datePlus(360) }
  );

  await execute(
    `INSERT INTO visas (id, student_id, visa_type, visa_no, issue_date, expiry_date, residence_permit_no,
document_url, status)
     VALUES (181, 181, 'X1', 'VISA-DEMO-001', '2026-01-10', :expiryDate, 'RP-DEMO-001',
'/uploads/visa/demo.pdf', 'valid')
     ON DUPLICATE KEY UPDATE student_id = 181, expiry_date = VALUES(expiry_date), status = 'valid'`,
    { expiryDate: datePlus(360) }
  );

  await execute(
    `INSERT INTO study_records
     (id, student_id, course_code, course_name, attendance_rate, homework_status, class_performance,
warning_level, teacher_id)
     VALUES (241, 181, 'CN-DEMO', 'Demo Chinese Language', 92.50, 'Completed', 'Normal', 'none', 121)
     ON DUPLICATE KEY UPDATE student_id = 181, attendance_rate = VALUES(attendance_rate), warning_level =
'none', teacher_id = 121`
  );

  const adapted = adaptGrade(
    { originalType: 'letter', originalGrade: 'A', originalCredit: 3 },
    { creditRatio: 1, passScore: 60, version: 'DEMO-RULE-2026' }
  );
  await execute(
    `INSERT INTO grade_records
     (id, student_id, course_code, course_name, source_school_id, source_country_code,
      original_grade, original_type, original_credit, adapted_score, adapted_gpa, adapted_credit,
      passed, rule_version, adapt_status, audit_status, audited_by, audited_at)
     VALUES (500, 181, 'CN-DEMO', 'Demo Chinese Language', 1, 'TH01',
      'A', 'letter', 3, :adaptedScore, :adaptedGpa, :adaptedCredit,
      1, :ruleVersion, 'success', 'approved', 1, NOW())
     ON DUPLICATE KEY UPDATE adapted_score = VALUES(adapted_score), adapted_gpa = VALUES(adapted_gpa),
      adapted_credit = VALUES(adapted_credit), adapt_status = 'success', audit_status = 'approved'`,
    adapted
  );
}

async function main() {
  await seedUsers();
  await seedNationalities();
  await seedTeachers();
  await seedClasses();
  await seedStudents();
  await seedDormitories();
  await seedVisas();
  await seedStudyRecords();
  await seedExternalSchools();
  await seedGradeRules();
  await seedGradeRecords();
  await seedDemoAliases();
  await pool.end();
  console.log('Business seed data inserted.');
}

main().catch(async (error) => {
  console.error(error);
  await pool.end();
  process.exit(1);
});
```

3.17 src/server.js

文件路径：src/server.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const path = require('path');
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
require('dotenv').config();

const authRoutes = require('./routes/auth');
const studentRoutes = require('./routes/students');
const teacherRoutes = require('./routes/teachers');
const gradeRoutes = require('./routes/grades');
const managementRoutes = require('./routes/management');

const app = express();

app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors());
app.use(express.json({ limit: '10mb' }));
app.get('/favicon.ico', (req, res) => {
  res.redirect(302, '/favicon.svg');
});
app.use(express.static(path.join(__dirname, '..', 'public')));

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', name: 'international-student-platform' });
});

app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/teachers', teacherRoutes);
app.use('/api/grades', gradeRoutes);
app.use('/api/management', managementRoutes);

app.use((req, res) => {
  res.status(404).json({ message: 'Not found' });
});

app.use((error, req, res, next) => {
  console.error(error);
  res.status(500).json({ message: 'Internal server error' });
});

if (require.main === module) {
  const port = Number(process.env.PORT || 3000);
  app.listen(port, () => {
    console.log(`Server running at http://***:${port}`);
  });
}

module.exports = app;
```

3.18 src/services/gradeAdapter.js

文件路径：src/services/gradeAdapter.js

文件职责：该文件属于面向国际留学生的信息管理系统V1.0的真实工程文件，用于支撑页面、接口、数据、配置、脚本或测试运行。

源码正文：

```
const LETTER_TO_PERCENT = {
  A: 95,
  'A-': 90,
  B: 85,
  'B-': 80,
  C: 75,
  'C-': 70,
  D: 65,
```

```
  F: 50
};

function gpaToPercent(value, scale) {
  const numeric = Number(value);
  const max = Number(scale || 4);
  if (Number.isNaN(numeric) || max <= 0) {
    throw new Error('Invalid GPA value');
  }
  return Math.round((numeric / max) * 100);
}

function toGpa4(percent) {
  if (percent >= 90) return 4.0;
  if (percent >= 85) return 3.7;
  if (percent >= 80) return 3.3;
  if (percent >= 75) return 3.0;
  if (percent >= 70) return 2.7;
  if (percent >= 65) return 2.3;
  if (percent >= 60) return 2.0;
  return 0;
}

function adaptGrade(input, rule = {}) {
  const type = String(input.originalType || '').toLowerCase();
  let percent;

  if (type === 'percent') {
    percent = Number(input.originalGrade);
  } else if (type === 'letter') {
    percent = LETTER_TO_PERCENT[String(input.originalGrade).trim().toUpperCase()];
  } else if (type === 'gpa') {
    percent = gpaToPercent(input.originalGrade, rule.gpaScale || input.gpaScale || 4);
  } else if (type === 'pass_fail') {
    percent = String(input.originalGrade).toLowerCase() === 'pass' ? 75 : 50;
  } else {
    throw new Error('Unsupported grade type');
  }

  if (typeof percent !== 'number' || Number.isNaN(percent) || percent < 0 || percent > 100) {
    throw new Error('Adapted grade is out of range');
  }

  const creditRatio = Number(rule.creditRatio || input.creditRatio || 1);
  return {
    adaptedScore: Math.round(percent),
    adaptedGpa: toGpa4(percent),
    adaptedCredit: Number((Number(input.originalCredit || 0) * creditRatio).toFixed(2)),
    passed: percent >= Number(rule.passScore || 60),
    ruleVersion: rule.version || 'default-v1'
  };
}

module.exports = { adaptGrade };
```

四.1 源码正文续页

.env.example（续页 1）

文件路径：.env.example

源码正文：

```
PORT=3000
DB_HOST=***
DB_PORT=3306
DB_USER=root
DB_PASSWORD=***
DB_NAME=international_student_platform
```

```
JWT_SECRET=***
JWT_EXPIRES_IN=8h
```

README.md（续页 1）

文件路径：README.md

源码正文：

```
# 面向国际留学生的信息管理系统 V1.0

前台使用 H5，后台使用 Node.js + Express，数据库使用 MySQL。

## 功能范围

- 学生登录与学生端中英双语界面
- 学生档案、国籍、班级、住宿、签证、学习情况基础数据模型
- 教师管理、教师班级管理、教师成绩管理入口
- 多国家、多学校成绩对接与成绩适配接口
- 成绩原始值、适配值、GPA、学分、规则版本、审核状态和异常原因留痕
- 管理端统计分析接口

## 数据库配置

默认连接配置：
```

host: *** port: 3306 user: root password: *** database: international_student_platform

```

## 启动步骤

1. 安装依赖：
```

npm install

```

2. 创建本地环境配置：
```

cp .env.example .env

```

3. 初始化数据库：
```

mysql -h *** -P 3306 -u root -p < database/schema.sql

```

也可以使用内置 Node 脚本初始化，不依赖本机 `mysql` 命令：
```

npm run db:init

```

4. 写入业务关联测试数据。该脚本会让每张业务表不少于 100 条记录，并保持用户、学生、教师、班级、住宿、签证、学习、外部学校、成绩规则和成绩记录之间的业务关联：
```

npm run db:seed

```

5. 启动服务：
```

npm start

```

访问地址：
```

http://***:3000

```

## 演示账号

三个演示账号的初始密码均为 `***`。
```

```
## 核心接口

- `POST /api/auth/login` 登录
- `GET /api/students/me` 学生查看本人档案
- `GET /api/students` 教师/管理员查看学生列表
- `GET /api/teachers` 管理员查看教师列表
- `POST /api/grades/adapt` 教师/管理员提交外部成绩并执行适配
- `GET /api/grades/student/:studentId` 查看学生成绩适配记录
- `PATCH /api/grades/:id/audit` 管理员审核成绩
- `GET /api/management/dashboard` 统计分析
- `GET /api/management/risks` 风险查询，聚合签证临期、成绩适配异常、低分风险、学习预警和住宿异常

## 开源协议

本项目使用 GNU General Public License v2.0（GPL-2.0）开源，详见 `LICENSE`。
```

database/schema.sql（续页 1）

文件路径：database/schema.sql

源码正文：

```
CREATE DATABASE IF NOT EXISTS international_student_platform
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

USE international_student_platform;

CREATE TABLE IF NOT EXISTS users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(64) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('student', 'teacher', 'admin') NOT NULL,
  display_name VARCHAR(120) NOT NULL,
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS nationalities (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  country_code VARCHAR(8) NOT NULL UNIQUE,
  name_zh VARCHAR(120) NOT NULL,
  name_en VARCHAR(120) NOT NULL,
  continent VARCHAR(60),
  primary_language VARCHAR(80),
  timezone VARCHAR(80)
);

CREATE TABLE IF NOT EXISTS teachers (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT UNIQUE,
  teacher_no VARCHAR(64) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  college VARCHAR(120),
  phone VARCHAR(40),
  email VARCHAR(120),
  teacher_type ENUM('course_teacher', 'head_teacher', 'counselor', 'admin_teacher') NOT NULL DEFAULT
'course_teacher',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_teachers_user FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS classes (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  code VARCHAR(64) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  college VARCHAR(120),
  major VARCHAR(120),
```

```
  teacher_id BIGINT,
  enrollment_year INT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_classes_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

CREATE TABLE IF NOT EXISTS students (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT UNIQUE,
  student_no VARCHAR(64) NOT NULL UNIQUE,
  chinese_name VARCHAR(120),
  english_name VARCHAR(120) NOT NULL,
  gender VARCHAR(20),
  birth_date DATE,
  nationality_id BIGINT,
  passport_no VARCHAR(80),
  passport_expiry DATE,
  phone VARCHAR(40),
  email VARCHAR(120),
  emergency_contact VARCHAR(160),
  enrollment_year INT,
  major VARCHAR(120),
  college VARCHAR(120),
  class_id BIGINT,
  status ENUM('active', 'suspended', 'withdrawn', 'graduated', 'delayed') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_students_user FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_students_nationality FOREIGN KEY (nationality_id) REFERENCES nationalities(id),
  CONSTRAINT fk_students_class FOREIGN KEY (class_id) REFERENCES classes(id)
);

CREATE TABLE IF NOT EXISTS dormitories (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  building VARCHAR(120) NOT NULL,
  room_no VARCHAR(60) NOT NULL,
  bed_no VARCHAR(60) NOT NULL,
  student_id BIGINT,
  check_in_date DATE,
  check_out_date DATE,
  status ENUM('available', 'occupied', 'reserved', 'maintenance') NOT NULL DEFAULT 'available',
  CONSTRAINT fk_dormitories_student FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS visas (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  visa_type VARCHAR(80) NOT NULL,
  visa_no VARCHAR(120),
  issue_date DATE,
  expiry_date DATE NOT NULL,
  residence_permit_no VARCHAR(120),
  document_url VARCHAR(255),
  status ENUM('valid', 'expiring', 'expired', 'renewing') NOT NULL DEFAULT 'valid',
  CONSTRAINT fk_visas_student FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS study_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  course_code VARCHAR(80),
  course_name VARCHAR(160) NOT NULL,
  attendance_rate DECIMAL(5,2),
  homework_status VARCHAR(120),
  class_performance VARCHAR(255),
  warning_level ENUM('none', 'notice', 'warning', 'serious') NOT NULL DEFAULT 'none',
  teacher_id BIGINT,
  recorded_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_study_records_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_study_records_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);
```

```
CREATE TABLE IF NOT EXISTS external_schools (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  country_code VARCHAR(8),
  name VARCHAR(180) NOT NULL,
  contact_name VARCHAR(120),
  contact_email VARCHAR(120),
  integration_type ENUM('api', 'excel', 'csv', 'manual', 'scheduled_sync') NOT NULL DEFAULT 'manual',
  endpoint_url VARCHAR(255),
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active'
);

CREATE TABLE IF NOT EXISTS grade_rules (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  school_id BIGINT,
  country_code VARCHAR(8),
  name VARCHAR(160) NOT NULL,
  version VARCHAR(40) NOT NULL,
  gpa_scale DECIMAL(4,2) NOT NULL DEFAULT 4.00,
  credit_ratio DECIMAL(6,3) NOT NULL DEFAULT 1.000,
  pass_score DECIMAL(5,2) NOT NULL DEFAULT 60.00,
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active',
  effective_from DATE,
  effective_to DATE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

package.json（续页 1）

文件路径：package.json

源码正文：

```
{
  "name": "southeast-asian-student-service-platform",
  "version": "1.0.0",
  "license": "GPL-2.0-only",
  "description": "Information management and service platform for Southeast Asian international students.",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js",
    "db:init": "node src/initDb.js",
    "db:seed": "node src/seedBusinessData.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

public/app.js（续页 1）

文件路径：public/app.js

源码正文：

```
const i18n = {
  zh: {
    systemTag: 'V1.0 留学生信息管理系统',
    systemName: '面向国际留学生的信息管理系统',
    logout: '退出',
    loginTitle: '用户登录',
    loginHint: '学生、教师和管理员使用统一入口登录。',
    username: '账号',
```

```
    password: '密码',
    loginButton: '登录系统',
    demoUsers: '演示账号',
    profile: '个人档案',
    grades: '成绩适配',
    learning: '学习情况',
    teacher: '教师管理',
    students: '学生管理',
    dashboard: '统计分析',
    risks: '风险查询',
    source: '来源',
    status: '状态',
    adaptedScore: '适配成绩',
    adaptedGpa: '适配 GPA',
    auditStatus: '审核状态',
    noData: '暂无数据',
    studentCount: '学生人数',
    teacherCount: '教师人数',
    visaWarnings: '签证预警',
    gradeExceptions: '成绩异常',
    loginFailed: '登录失败，请检查账号或密码。',
    loadFailed: '数据加载失败，请确认后台和数据库已启动。',
    studentOnly: '学生端支持中英文切换并仅展示本人数据。',
    teacherScope: '教师端用于班级、课程、成绩录入与学习预警管理。',
    adminScope: '管理员端用于档案、国籍、班级、住宿、签证、教师、成绩规则和统计分析管理。'
  },
  en: {
    systemTag: 'V1.0 International Student Information System',
    systemName: 'Information Management System for International Students',
    logout: 'Logout',
    loginTitle: 'Sign In',
    loginHint: 'Students, teachers, and administrators use the same portal.',
    username: 'Username',
    password: 'Password',
    loginButton: 'Sign In',
    demoUsers: 'Demo accounts',
    profile: 'Profile',
    grades: 'Grade Adapter',
    learning: 'Learning',
    teacher: 'Teacher Management',
    students: 'Student Management',
    dashboard: 'Analytics',
    risks: 'Risk Query',
    source: 'Source',
    status: 'Status',
    adaptedScore: 'Adapted Score',
    adaptedGpa: 'Adapted GPA',
    auditStatus: 'Audit Status',
    noData: 'No data',
    studentCount: 'Students',
    teacherCount: 'Teachers',
    visaWarnings: 'Visa Alerts',
    gradeExceptions: 'Grade Exceptions',
    loginFailed: 'Sign-in failed. Check your username or password.',
    loadFailed: 'Failed to load data. Confirm the API and database are running.',
    studentOnly: 'The student client supports Chinese and English and only shows personal data.',
    teacherScope: 'The teacher portal supports classes, courses, grade entry, and learning alerts.',
    adminScope: 'The admin portal manages profiles, nationalities, classes, dormitories, visas, teachers,
grade rules, and analytics.'
  }
};

let lang = localStorage.getItem('lang') || 'zh';
let session = JSON.parse(localStorage.getItem('session') || 'null');
let currentTab = 'profile';

const loginPanel = document.querySelector('#loginPanel');
const workspace = document.querySelector('#workspace');
const tabs = document.querySelector('#tabs');
const summaryGrid = document.querySelector('#summaryGrid');
```

```
const primaryView = document.querySelector('#primaryView');
const sideView = document.querySelector('#sideView');
const loginMessage = document.querySelector('#loginMessage');
const logoutBtn = document.querySelector('#logoutBtn');

function t(key) {
  return i18n[lang][key] || key;
}

function applyLanguage() {
  document.documentElement.lang = lang === 'zh' ? 'zh-CN' : 'en';
  document.querySelectorAll('[data-i18n]').forEach((node) => {
    node.textContent = t(node.dataset.i18n);
  });
  if (session) renderApp();
}

async function api(path, options = {}) {
  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };
  if (session?.token) headers.Authorization = `Bearer ${session.token}`;
  const response = await fetch(path, { ...options, headers });
  if (!response.ok) {
    const body = await response.json().catch(() => ({}));
    throw new Error(body.message || response.statusText);
  }
  return response.json();
}

function setLoggedIn(value) {
  loginPanel.classList.toggle('hidden', value);
  workspace.classList.toggle('hidden', !value);
  logoutBtn.classList.toggle('hidden', !value);
}

function roleTabs(role) {
  if (role === 'student') return ['profile', 'grades', 'learning'];
  if (role === 'teacher') return ['dashboard', 'students', 'grades', 'risks'];
  return ['dashboard', 'students', 'teacher', 'grades', 'risks'];
}

function renderTabs() {
  tabs.innerHTML = roleTabs(session.user.role)
    .map((tab) => `<button class="tab-button ${tab === currentTab ? 'active' : ''}" data-
tab="${tab}">${t(tab)}</button>`)
    .join('');
  tabs.querySelectorAll('button').forEach((button) => {
    button.addEventListener('click', () => {
      currentTab = button.dataset.tab;
      renderApp();
    });
  });
}

function metric(label, value, tone = '') {
  return `<div class="metric"><span class="muted">${label}</span><strong class="${tone}">${value}</strong>
</div>`;
}

function table(headers, rows) {
  if (!rows.length) return `<p class="muted">${t('noData')}</p>`;
  return `
    <table class="data-table">
```

public/index.html（续页 1）

文件路径：public/index.html

源码正文：

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>International Student Information Platform</title>
    <link rel="icon" href="/favicon.svg" type="image/svg+xml" />
    <link rel="stylesheet" href="/styles.css" />
  </head>
  <body>
    <header class="topbar">
      <div>
        <p class="eyebrow" data-i18n="systemTag">V1.0 留学生信息管理系统</p>
        <h1 data-i18n="systemName">面向国际留学生的信息管理系统</h1>
      </div>
      <div class="toolbar">
        <button class="icon-button" id="langToggle" title="Language">中 / EN</button>
        <button class="ghost-button hidden" id="logoutBtn" data-i18n="logout">退出</button>
      </div>
    </header>

    <main class="layout">
      <section class="login-panel" id="loginPanel">
        <div class="panel-title">
          <h2 data-i18n="loginTitle">用户登录</h2>
          <p data-i18n="loginHint">学生、教师和管理员使用统一入口登录。</p>
        </div>
        <form id="loginForm" class="form-grid">
          <label>
            <span data-i18n="username">账号</span>
            <input name="username" autocomplete="username" value="student01" required />
          </label>
          <label>
            <span data-i18n="password">密码</span>
            <input name="password" type="password" autocomplete="current-password" value="***" required />
          </label>
          <button class="primary-button" type="submit" data-i18n="loginButton">登录系统</button>
          <p class="message" id="loginMessage"></p>
        </form>
        <div class="demo-users">
          <span data-i18n="demoUsers">演示账号</span>
          <code>student01 / ******</code>
          <code>teacher01 / ******</code>
          <code>student001 / ******</code>
          <code>admin / ******</code>
        </div>
      </section>

      <section class="workspace hidden" id="workspace">
        <nav class="tabs" id="tabs"></nav>

        <section class="summary-grid" id="summaryGrid"></section>

        <section class="content-grid">
          <article class="surface" id="primaryView"></article>
          <aside class="surface compact" id="sideView"></aside>
        </section>
      </section>
    </main>

    <script src="/app.js"></script>
  </body>
</html>
```

public/styles.css（续页 1）

文件路径：public/styles.css

源码正文：

```
:root {
  color-scheme: light;
  --bg: #f6f7f9;
  --surface: #ffffff;
  --surface-strong: #eef4f1;
  --line: #d8dee6;
  --text: #1f2933;
  --muted: #64707d;
  --accent: #1f7a6d;
  --accent-dark: #14594f;
  --warning: #b45f06;
  --danger: #b42318;
  --blue: #2f5f98;
  --shadow: 0 14px 34px rgba(21, 39, 60, 0.08);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background: var(--bg);
  color: var(--text);
  font-family: Arial, "Microsoft YaHei", sans-serif;
}

button,
input,
select {
  font: inherit;
}

.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  padding: 24px clamp(16px, 4vw, 48px);
  background: #ffffff;
  border-bottom: 1px solid var(--line);
}

.eyebrow {
  margin: 0 0 6px;
  color: var(--accent);
  font-size: 13px;
  font-weight: 700;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 0;
  font-size: clamp(22px, 3vw, 34px);
}

h2 {
  margin-bottom: 8px;
  font-size: 22px;
}

h3 {
```

```
  margin-bottom: 12px;
  font-size: 17px;
}

.toolbar,
.tabs,
.actions {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.layout {
  width: min(1180px, calc(100% - 32px));
  margin: 28px auto 48px;
}

.login-panel,
.surface {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 8px;
  box-shadow: var(--shadow);
}

.login-panel {
  width: min(520px, 100%);
  padding: 26px;
}

.panel-title p,
.muted {
  color: var(--muted);
}

.form-grid {
  display: grid;
  gap: 16px;
}

label {
  display: grid;
  gap: 6px;
  color: var(--muted);
  font-size: 14px;
}

input,
select {
  width: 100%;
  min-height: 42px;
  padding: 9px 11px;
  border: 1px solid var(--line);
  border-radius: 6px;
  background: #ffffff;
  color: var(--text);
}

.primary-button,
.ghost-button,
.icon-button,
.tab-button {
  min-height: 40px;
  border-radius: 6px;
  border: 1px solid transparent;
  cursor: pointer;
}
```

```
.primary-button {
  background: var(--accent);
```

src/config/db.js（续页 1）

文件路径：src/config/db.js

源码正文：

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST || '***',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '***',
  database: process.env.DB_NAME || 'international_student_platform',
  waitForConnections: true,
  connectionLimit: 10,
  namedPlaceholders: true
});

module.exports = pool;
```

src/initDb.js（续页 1）

文件路径：src/initDb.js

源码正文：

```
const fs = require('fs');
const path = require('path');
const mysql = require('mysql2/promise');
require('dotenv').config();

async function initDb() {
  const sql = fs.readFileSync(path.join(__dirname, '..', 'database', 'schema.sql'), 'utf8');
  const connection = await mysql.createConnection({
    host: process.env.DB_HOST || '***',
    port: Number(process.env.DB_PORT || 3306),
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '***',
    multipleStatements: true
  });

  await connection.query(sql);
  await connection.end();
  console.log('Database initialized.');
}

initDb().catch((error) => {
  console.error(error.message);
  process.exit(1);
});
```

src/middleware/auth.js（续页 1）

文件路径：src/middleware/auth.js

源码正文：

```
const jwt = require('jsonwebtoken');

function auth(requiredRoles = []) {
  return (req, res, next) => {
    const header = req.headers.authorization || '';
    const token = header.startsWith('Bearer ') ? header.slice(7) : '';

    if (!token) {
      return res.status(401).json({ message: 'Missing authorization token' });
    }
```

```
    try {
      const payload = jwt.verify(token, process.env.JWT_SECRET || '***');
      if (requiredRoles.length && !requiredRoles.includes(payload.role)) {
        return res.status(403).json({ message: 'Permission denied' });
      }
      req.user = payload;
      next();
    } catch (error) {
      return res.status(401).json({ message: 'Invalid or expired token' });
    }
  };
}

module.exports = auth;
```

src/routes/auth.js（续页 1）

文件路径：src/routes/auth.js

源码正文：

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../config/db');

const router = express.Router();

router.post('/login', async (req, res, next) => {
  try {
    const { username, password } = req.body;
    const [rows] = await pool.execute(
      'SELECT id, username, password_hash, role, display_name, status FROM users WHERE username = :username
LIMIT 1',
      { username }
    );

    const user = rows[0];
    if (!user || user.status !== 'active') {
      return res.status(401).json({ message: 'Invalid username or password' });
    }

    const ok = await bcrypt.compare(password, user.password_hash);
    if (!ok) {
      return res.status(401).json({ message: 'Invalid username or password' });
    }

    const token = jwt.sign(
      { id: user.id, username: user.username, role: user.role, name: user.display_name },
      process.env.JWT_SECRET || '***',
      { expiresIn: process.env.JWT_EXPIRES_IN || '8h' }
    );

    res.json({
      token,
      user: {
        id: user.id,
        username: user.username,
        role: user.role,
        displayName: user.display_name
      }
    });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

src/routes/grades.js（续页 1）

文件路径：src/routes/grades.js

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');
const { adaptGrade } = require('../services/gradeAdapter');

const router = express.Router();

async function findRule(schoolId, countryCode) {
  const [rows] = await pool.execute(
    `SELECT *
     FROM grade_rules
     WHERE (school_id = :schoolId OR school_id IS NULL)
       AND (country_code = :countryCode OR country_code IS NULL)
       AND status = 'active'
     ORDER BY school_id DESC, id DESC
     LIMIT 1`,
    { schoolId: schoolId || null, countryCode: countryCode || null }
  );
  const rule = rows[0];
  if (!rule) return {};

  return {
    gpaScale: rule.gpa_scale,
    creditRatio: rule.credit_ratio,
    passScore: rule.pass_score,
    version: rule.version
  };
}

router.get('/', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT gr.*, s.student_no, s.english_name, s.chinese_name, es.name AS source_school_name
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       LEFT JOIN external_schools es ON es.id = gr.source_school_id
       ORDER BY gr.id DESC
       LIMIT 200`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/adapt', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const input = req.body;
    const rule = await findRule(input.schoolId, input.countryCode);
    const adapted = adaptGrade(input, rule);

    const [result] = await pool.execute(
      `INSERT INTO grade_records
       (student_id, course_code, course_name, source_school_id, source_country_code,
        original_grade, original_type, original_credit, adapted_score, adapted_gpa,
        adapted_credit, passed, rule_version, adapt_status, audit_status)
       VALUES (:studentId, :courseCode, :courseName, :schoolId, :countryCode,
        :originalGrade, :originalType, :originalCredit, :adaptedScore, :adaptedGpa,
        :adaptedCredit, :passed, :ruleVersion, 'success', 'pending')`,
      {
        studentId: input.studentId,
        courseCode: input.courseCode,
        courseName: input.courseName,
        schoolId: input.schoolId || null,
        countryCode: input.countryCode || null,
```

```
        originalGrade: String(input.originalGrade),
        originalType: input.originalType,
        originalCredit: input.originalCredit || 0,
        adaptedScore: adapted.adaptedScore,
        adaptedGpa: adapted.adaptedGpa,
        adaptedCredit: adapted.adaptedCredit,
        passed: adapted.passed ? 1 : 0,
        ruleVersion: adapted.ruleVersion
      }
    );

    res.status(201).json({ id: result.insertId, ...adapted, auditStatus: 'pending' });
  } catch (error) {
    if (req.body && req.body.studentId) {
      await pool.execute(
        `INSERT INTO grade_records
         (student_id, course_code, course_name, source_school_id, source_country_code,
          original_grade, original_type, original_credit, adapt_status, exception_reason, audit_status)
         VALUES (:studentId, :courseCode, :courseName, :schoolId, :countryCode,
          :originalGrade, :originalType, :originalCredit, 'exception', :reason, 'pending')`,
        {
          studentId: req.body.studentId,
          courseCode: req.body.courseCode || null,
          courseName: req.body.courseName || null,
          schoolId: req.body.schoolId || null,
          countryCode: req.body.countryCode || null,
          originalGrade: String(req.body.originalGrade || ''),
          originalType: req.body.originalType || null,
          originalCredit: req.body.originalCredit || 0,
          reason: error.message
        }
      );
    }
    res.status(422).json({ message: error.message });
  }
});

router.get('/student/:studentId', auth(['admin', 'teacher', 'student']), async (req, res, next) => {
  try {
    if (req.user.role === 'student') {
      const [owned] = await pool.execute('SELECT id FROM students WHERE user_id = :userId', { userId:
req.user.id });
      if (!owned[0] || String(owned[0].id) !== String(req.params.studentId)) {
        return res.status(403).json({ message: 'Permission denied' });
      }
    }

    const [rows] = await pool.execute(
      `SELECT gr.*, es.name AS source_school_name
       FROM grade_records gr
       LEFT JOIN external_schools es ON es.id = gr.source_school_id
       WHERE gr.student_id = :studentId
       ORDER BY gr.id DESC`,
      { studentId: req.params.studentId }
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.patch('/:id/audit', auth(['admin']), async (req, res, next) => {
  try {
    const { auditStatus } = req.body;
    await pool.execute(
      'UPDATE grade_records SET audit_status = :auditStatus, audited_by = :auditedBy, audited_at = NOW()
WHERE id = :id',
      { auditStatus, auditedBy: req.user.id, id: req.params.id }
    );
    res.json({ id: Number(req.params.id), auditStatus });
```

```
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

src/routes/management.js（续页 1）

文件路径：src/routes/management.js

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/dashboard', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [[students]] = await pool.execute('SELECT COUNT(*) AS count FROM students');
    const [[teachers]] = await pool.execute('SELECT COUNT(*) AS count FROM teachers');
    const [[visaWarnings]] = await pool.execute(
      'SELECT COUNT(*) AS count FROM visas WHERE expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
INTERVAL 60 DAY)'
    );
    const [[gradeExceptions]] = await pool.execute(
      "SELECT COUNT(*) AS count FROM grade_records WHERE adapt_status = 'exception'"
    );

    res.json({
      students: students.count,
      teachers: teachers.count,
      visaWarnings: visaWarnings.count,
      gradeExceptions: gradeExceptions.count
    });
  } catch (error) {
    next(error);
  }
});

router.get('/nationalities', auth(['admin', 'teacher', 'student']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute('SELECT * FROM nationalities ORDER BY name_en');
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.get('/classes', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT c.*, t.name AS teacher_name
       FROM classes c
       LEFT JOIN teachers t ON t.id = c.teacher_id
       ORDER BY c.id DESC`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.get('/risks', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [visaRisks] = await pool.execute(
      `SELECT 'visa_expiring' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              v.expiry_date AS risk_date,
              CONCAT('Visa expires on ', DATE_FORMAT(v.expiry_date, '%Y-%m-%d')) AS detail
```

```
       FROM visas v
       JOIN students s ON s.id = v.student_id
       WHERE v.expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 60 DAY)
       ORDER BY v.expiry_date ASC
       LIMIT 100`
    );

    const [gradeRisks] = await pool.execute(
      `SELECT 'grade_exception' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              gr.created_at AS risk_date,
              CONCAT(COALESCE(gr.course_name, 'Unknown course'), ': ', COALESCE(gr.exception_reason,
gr.adapt_status)) AS detail
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       WHERE gr.adapt_status = 'exception'
       ORDER BY gr.created_at DESC
       LIMIT 100`
    );

    const [lowScoreRisks] = await pool.execute(
      `SELECT 'low_score' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              gr.created_at AS risk_date,
              CONCAT(COALESCE(gr.course_name, 'Unknown course'), ': adapted score ', gr.adapted_score) AS
detail
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       WHERE gr.adapted_score IS NOT NULL AND gr.adapted_score < 60
       ORDER BY gr.adapted_score ASC
       LIMIT 100`
    );

    const [learningRisks] = await pool.execute(
      `SELECT 'learning_warning' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              sr.recorded_at AS risk_date,
              CONCAT(sr.course_name, ': ', sr.warning_level, ', attendance ', COALESCE(sr.attendance_rate,
0), '%') AS detail
       FROM study_records sr
       JOIN students s ON s.id = sr.student_id
       WHERE sr.warning_level IN ('warning', 'serious') OR sr.attendance_rate < 70
       ORDER BY sr.recorded_at DESC
       LIMIT 100`
    );

    const [dormitoryRisks] = await pool.execute(
      `SELECT 'dormitory_exception' AS risk_type, s.student_no, s.english_name, s.chinese_name,
              d.check_out_date AS risk_date,
              CONCAT(d.building, '-', d.room_no, '-', d.bed_no, ': ', d.status) AS detail
       FROM dormitories d
       JOIN students s ON s.id = d.student_id
       WHERE d.status IN ('maintenance', 'reserved') OR d.check_out_date < CURDATE()
       ORDER BY d.check_out_date ASC
       LIMIT 100`
    );

    const items = [...visaRisks, ...gradeRisks, ...lowScoreRisks, ...learningRisks, ...dormitoryRisks];
    const summary = items.reduce((acc, item) => {
      acc[item.risk_type] = (acc[item.risk_type] || 0) + 1;
      return acc;
    }, {});

    res.json({ summary, items });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

src/routes/students.js（续页 1）

文件路径：src/routes/students.js

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/me', auth(['student']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT s.*, c.name AS class_name, n.name_zh AS nationality_zh, n.name_en AS nationality_en
       FROM students s
       LEFT JOIN classes c ON c.id = s.class_id
       LEFT JOIN nationalities n ON n.id = s.nationality_id
       WHERE s.user_id = :userId`,
      { userId: req.user.id }
    );
    res.json(rows[0] || null);
  } catch (error) {
    next(error);
  }
});

router.get('/', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT s.id, s.student_no, s.chinese_name, s.english_name, s.gender, s.enrollment_year,
              s.status, c.name AS class_name, n.name_zh AS nationality
       FROM students s
       LEFT JOIN classes c ON c.id = s.class_id
       LEFT JOIN nationalities n ON n.id = s.nationality_id
       ORDER BY s.id DESC`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/', auth(['admin']), async (req, res, next) => {
  try {
    const data = req.body;
    const [result] = await pool.execute(
      `INSERT INTO students
       (user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
        passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year,
        major, college, class_id, status)
       VALUES (:userId, :studentNo, :chineseName, :englishName, :gender, :birthDate, :nationalityId,
        :passportNo, :passportExpiry, :phone, :email, :emergencyContact, :enrollmentYear,
        :major, :college, :classId, :status)`,
      {
        userId: data.userId || null,
        studentNo: data.studentNo,
        chineseName: data.chineseName || null,
        englishName: data.englishName,
        gender: data.gender || null,
        birthDate: data.birthDate || null,
        nationalityId: data.nationalityId || null,
        passportNo: data.passportNo || null,
        passportExpiry: data.passportExpiry || null,
        phone: data.phone || null,
        email: data.email || null,
        emergencyContact: data.emergencyContact || null,
        enrollmentYear: data.enrollmentYear || null,
        major: data.major || null,
        college: data.college || null,
```

```
        classId: data.classId || null,
        status: data.status || 'active'
      }
    );
    res.status(201).json({ id: result.insertId });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

src/routes/teachers.js（续页 1）

文件路径：src/routes/teachers.js

源码正文：

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/', auth(['admin']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT t.*, u.username
       FROM teachers t
       LEFT JOIN users u ON u.id = t.user_id
       ORDER BY t.id DESC`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.get('/me/classes', auth(['teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT c.*
       FROM classes c
       JOIN teachers t ON t.id = c.teacher_id
       WHERE t.user_id = :userId`,
      { userId: req.user.id }
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/', auth(['admin']), async (req, res, next) => {
  try {
    const data = req.body;
    const [result] = await pool.execute(
      `INSERT INTO teachers (user_id, teacher_no, name, college, phone, email, teacher_type)
       VALUES (:userId, :teacherNo, :name, :college, :phone, :email, :teacherType)`,
      {
        userId: data.userId || null,
        teacherNo: data.teacherNo,
        name: data.name,
        college: data.college || null,
        phone: data.phone || null,
        email: data.email || null,
        teacherType: data.teacherType || 'course_teacher'
      }
    );
    res.status(201).json({ id: result.insertId });
  } catch (error) {
```

```
    next(error);
  }
});

module.exports = router;
```

src/seedBusinessData.js（续页 1）

文件路径：src/seedBusinessData.js

源码正文：

```
const bcrypt = require('bcryptjs');
const pool = require('./config/db');
const { adaptGrade } = require('./services/gradeAdapter');

const passwordHash = bcrypt.hashSync('***', 10);
const countryCodes = [
  'TH', 'VN', 'MY', 'ID', 'PH', 'SG', 'LA', 'KH', 'MM', 'BN',
  'CN', 'JP', 'KR', 'IN', 'PK', 'BD', 'LK', 'NP', 'MN', 'KZ'
];
const gradeTypes = ['percent', 'letter', 'gpa', 'pass_fail'];
const letters = ['A', 'A-', 'B', 'B-', 'C', 'C-', 'D', 'F'];

function pad(num, size = 3) {
  return String(num).padStart(size, '0');
}

function datePlus(days) {
  const date = new Date();
  date.setDate(date.getDate() + days);
  return date.toISOString().slice(0, 10);
}

async function execute(sql, params = {}) {
  await pool.execute(sql, params);
}

async function seedUsers() {
  await execute(
    `INSERT INTO users (id, username, password_hash, role, display_name, status)
     VALUES (1, 'admin', :passwordHash, 'admin', 'System Admin', 'active')
     ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), status = 'active'`,
    { passwordHash }
  );

  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO users (id, username, password_hash, role, display_name, status)
       VALUES (:id, :username, :passwordHash, :role, :displayName, 'active')
       ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), role = VALUES(role), display_name =
VALUES(display_name), status = 'active'`,
      {
        id: 1000 + i,
        username: `teacher${pad(i)}`,
        passwordHash,
        role: 'teacher',
        displayName: `Teacher ${pad(i)}`
      }
    );
  }

  for (let i = 1; i <= 180; i += 1) {
    await execute(
      `INSERT INTO users (id, username, password_hash, role, display_name, status)
       VALUES (:id, :username, :passwordHash, 'student', :displayName, 'active')
       ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), role = 'student', display_name =
VALUES(display_name), status = 'active'`,
      {
        id: 2000 + i,
        username: `student${pad(i)}`,
```

```
        passwordHash,
        displayName: `International Student ${pad(i)}`
      }
    );
  }
}

async function seedNationalities() {
  for (let i = 1; i <= 120; i += 1) {
    const code = `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i / countryCodes.length),
2)}`;
    await execute(
      `INSERT INTO nationalities (id, country_code, name_zh, name_en, continent, primary_language,
timezone)
       VALUES (:id, :countryCode, :nameZh, :nameEn, 'Asia-Pacific', :language, :timezone)
       ON DUPLICATE KEY UPDATE name_zh = VALUES(name_zh), name_en = VALUES(name_en), primary_language =
VALUES(primary_language), timezone = VALUES(timezone)`,
      {
        id: i,
        countryCode: code,
        nameZh: `国际生来源地${pad(i)}`,
        nameEn: `International Region ${pad(i)}`,
        language: `Language ${pad(((i - 1) % 20) + 1, 2)}`,
        timezone: `Asia/Test-${pad(((i - 1) % 24) + 1, 2)}`
      }
    );
  }
}

async function seedTeachers() {
  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
       VALUES (:id, :userId, :teacherNo, :name, :college, :phone, :email, :teacherType)
       ON DUPLICATE KEY UPDATE user_id = VALUES(user_id), name = VALUES(name), college = VALUES(college),
phone = VALUES(phone), email = VALUES(email), teacher_type = VALUES(teacher_type)`,
      {
        id: i,
        userId: 1000 + i,
        teacherNo: `T2026${pad(i)}`,
        name: `Teacher ${pad(i)}`,
        college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
        phone: `1380000${pad(i, 4)}`,
        email: `teacher${pad(i)}@example.edu`,
        teacherType: ['course_teacher', 'head_teacher', 'counselor', 'admin_teacher'][i % 4]
      }
    );
  }
}

async function seedClasses() {
  for (let i = 1; i <= 120; i += 1) {
    await execute(
      `INSERT INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
       VALUES (:id, :code, :name, :college, :major, :teacherId, :year)
       ON DUPLICATE KEY UPDATE name = VALUES(name), college = VALUES(college), major = VALUES(major),
teacher_id = VALUES(teacher_id), enrollment_year = VALUES(enrollment_year)`,
      {
        id: i,
        code: `ISE-2026-${pad(i)}`,
        name: `International Student Class ${pad(i)}`,
        college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
        major: ['Chinese Language', 'Business Chinese', 'Computer Science', 'International Trade'][i % 4],
        teacherId: ((i - 1) % 120) + 1,
        year: 2024 + (i % 3)
      }
    );
  }
}
```

```
async function seedStudents() {
  for (let i = 1; i <= 180; i += 1) {
    await execute(
      `INSERT INTO students
       (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
        passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, college,
class_id, status)
       VALUES (:id, :userId, :studentNo, :chineseName, :englishName, :gender, :birthDate, :nationalityId,
        :passportNo, :passportExpiry, :phone, :email, :emergencyContact, :enrollmentYear, :major, :college,
:classId, :status)
       ON DUPLICATE KEY UPDATE user_id = VALUES(user_id), chinese_name = VALUES(chinese_name), english_name
= VALUES(english_name),
        nationality_id = VALUES(nationality_id), class_id = VALUES(class_id), status = VALUES(status)`,
      {
        id: i,
        userId: 2000 + i,
        studentNo: `S2026${pad(i)}`,
        chineseName: `留学生${pad(i)}`,
        englishName: `International Student ${pad(i)}`,
        gender: i % 2 === 0 ? 'Male' : 'Female',
        birthDate: `200${i % 6}-0${(i % 9) + 1}-15`,
```

src/server.js（续页 1）

文件路径：src/server.js

源码正文：

```
const path = require('path');
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
require('dotenv').config();

const authRoutes = require('./routes/auth');
const studentRoutes = require('./routes/students');
const teacherRoutes = require('./routes/teachers');
const gradeRoutes = require('./routes/grades');
const managementRoutes = require('./routes/management');

const app = express();

app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors());
app.use(express.json({ limit: '10mb' }));
app.get('/favicon.ico', (req, res) => {
  res.redirect(302, '/favicon.svg');
});
app.use(express.static(path.join(__dirname, '..', 'public')));

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', name: 'international-student-platform' });
});

app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/teachers', teacherRoutes);
app.use('/api/grades', gradeRoutes);
app.use('/api/management', managementRoutes);

app.use((req, res) => {
  res.status(404).json({ message: 'Not found' });
});

app.use((error, req, res, next) => {
  console.error(error);
  res.status(500).json({ message: 'Internal server error' });
});

if (require.main === module) {
  const port = Number(process.env.PORT || 3000);
```

```
  app.listen(port, () => {
    console.log(`Server running at http://***:${port}`);
  });
}

module.exports = app;
```

src/services/gradeAdapter.js（续页 1）

文件路径：src/services/gradeAdapter.js

源码正文：

```
const LETTER_TO_PERCENT = {
  A: 95,
  'A-': 90,
  B: 85,
  'B-': 80,
  C: 75,
  'C-': 70,
  D: 65,
  F: 50
};

function gpaToPercent(value, scale) {
  const numeric = Number(value);
  const max = Number(scale || 4);
  if (Number.isNaN(numeric) || max <= 0) {
    throw new Error('Invalid GPA value');
  }
  return Math.round((numeric / max) * 100);
}

function toGpa4(percent) {
  if (percent >= 90) return 4.0;
  if (percent >= 85) return 3.7;
  if (percent >= 80) return 3.3;
  if (percent >= 75) return 3.0;
  if (percent >= 70) return 2.7;
  if (percent >= 65) return 2.3;
  if (percent >= 60) return 2.0;
  return 0;
}

function adaptGrade(input, rule = {}) {
  const type = String(input.originalType || '').toLowerCase();
  let percent;

  if (type === 'percent') {
    percent = Number(input.originalGrade);
  } else if (type === 'letter') {
    percent = LETTER_TO_PERCENT[String(input.originalGrade).trim().toUpperCase()];
  } else if (type === 'gpa') {
    percent = gpaToPercent(input.originalGrade, rule.gpaScale || input.gpaScale || 4);
  } else if (type === 'pass_fail') {
    percent = String(input.originalGrade).toLowerCase() === 'pass' ? 75 : 50;
  } else {
    throw new Error('Unsupported grade type');
  }

  if (typeof percent !== 'number' || Number.isNaN(percent) || percent < 0 || percent > 100) {
    throw new Error('Adapted grade is out of range');
  }

  const creditRatio = Number(rule.creditRatio || input.creditRatio || 1);
  return {
    adaptedScore: Math.round(percent),
    adaptedGpa: toGpa4(percent),
    adaptedCredit: Number((Number(input.originalCredit || 0) * creditRatio).toFixed(2)),
    passed: percent >= Number(rule.passScore || 60),
    ruleVersion: rule.version || 'default-v1'
```

```
  };
}

module.exports = { adaptGrade };
```